

ĐỀ TÀI

NỀN TẢNG KẾT NỐI CỘNG ĐỒNG
NHIẾP ẢNH PHIM VỚI CÁC DỊCH VỤ
PHÒNG TỐI VÀ PHÒNG CHỤP

Giảng viên hướng dẫn: Nguyễn Văn Chiến

Thành viên nhóm:

- Nguyễn Khải Anh Thơ
- Nguyễn Thị Vân Anh
- Trần Thị Thanh Ngọc
- Huỳnh Đức Huy
- Hoàng Thành Công

Mục lục

1	DẪN NHẬP ĐỀ TÀI	4
1.1	Bối cảnh thực tiễn	4
1.2	Vấn đề đặt ra cho bài toán dữ liệu	4
1.3	Mục tiêu đề án	5
1.4	Quy trình nghiệp vụ tổng quan	5
2	PHÂN TÍCH YÊU CẦU HỆ THỐNG	7
2.1	Yêu cầu chức năng theo tác nhân	7
2.2	Sáu luồng nghiệp vụ cốt lõi	8
3	THIẾT KẾ MỨC QUAN NIỆM	10
3.1	Danh mục thực thể và ý nghĩa	10
3.2	Mô tả thuộc tính từng thực thể	12
3.2.1	USERS	12
3.2.2	SPACES	12
3.2.3	RESOURCES	13
3.2.4	MAINTENANCE_SCHEDULES	14
3.2.5	BOOKINGS	15
3.2.6	BOOKING_RESOURCES	16
3.2.7	REVIEWS	16
3.2.8	PHOTO_COLLECTIONS	17
3.2.9	PHOTOS	17
3.2.10	POSTS	17
3.2.11	WORKSHOPS	18
3.2.12	WORKSHOP_REGISTRATION	18
3.2.13	SERVICE_PACKAGES	19
3.2.14	PACKAGE_ITEMS	20
3.2.15	CATEGORIES	20
3.2.16	SPACE_CATEGORIES	20
3.2.17	RESOURCE_CATEGORIES	21
3.2.18	POST_CATEGORIES	21

3.2.19	PROMOTIONS	21
3.2.20	TRANSACTIONS	22
3.2.21	COMPLAINTS	23
3.2.22	CHAT_LOGS	23
3.2.23	FAVORITES	24
3.2.24	NOTIFICATIONS	24
3.3	Ma trận quan hệ giữa các thực thể	25
3.4	Quy tắc nghiệp vụ (Business Rules)	26
3.5	Sơ đồ ERD	28
4	THIẾT KẾ MỨC LOGIC	29
4.1	Lược đồ quan hệ	29
4.2	Chuẩn hóa dữ liệu	31
4.2.1	Kiểm tra 1NF	31
4.2.2	Kiểm tra 2NF	32
4.2.3	Kiểm tra 3NF	32
4.2.4	Phân tích phụ thuộc hàm và BCNF	33
5	TRIỂN KHAI CSDL VẬT LÝ	34
5.1	Môi trường và công cụ	34
5.2	Script DDL — Tạo bảng	34
5.2.1	Phần A — Gói dịch vụ	34
5.2.2	Phần B — Danh mục và ba bảng trung gian	34
5.2.3	Phần C — Khuyến mãi	35
5.2.4	Phần D — Giao dịch thanh toán	36
5.2.5	Phần E — Khiếu nại	36
5.2.6	Phần F — Nhật ký trợ lý AI	37
5.3	Script ràng buộc — Constraint	37
5.3.1	Ràng buộc khóa ngoại (FOREIGN KEY)	37
5.3.2	Ràng buộc duy nhất (UNIQUE)	37
5.3.3	Ràng buộc miền giá trị (CHECK)	38
5.3.4	Ràng buộc mặc định (DEFAULT)	38
5.4	Trigger nghiệp vụ	39
5.4.1	Trigger đồng bộ số lượng đăng ký workshop khi đổi trạng thái	39
5.4.2	Trigger tăng số lượt dùng khuyến mãi khi tạo đơn đặt chỗ	39
5.4.3	Trigger đồng bộ số lượt dùng khuyến mãi khi sửa đơn đặt chỗ	40
5.4.4	Trigger chặn trùng lịch không gian khi tạo đơn đặt chỗ	40
5.4.5	Trigger chặn trùng lịch không gian khi cập nhật đơn đặt chỗ	42
5.4.6	Trigger chặn trùng lịch thiết bị khi thêm thiết bị vào đơn đặt chỗ	43
5.4.7	Trigger tăng số lượng đăng ký workshop khi thêm lượt đăng ký	44

5.4.8	Trigger giảm số lượng đăng ký workshop khi xóa lượt đăng ký	44
5.4.9	Trigger chặn đăng ký workshop khi đã đủ chỗ hoặc đã diễn ra	45
5.5	Chiến lược nạp dữ liệu mẫu	46
5.6	Index tối ưu truy vấn	46
6	KIỂM THỬ VÀ MINH HỌA TRUY VẤN	47
6.1	Nguyên tắc nạp dữ liệu	47
6.2	Kiểm thử	47
6.3	Bộ truy vấn nghiệp vụ minh họa	48
6.3.1	Nhóm truy vấn người dùng & phân quyền	48
6.3.2	Nhóm truy vấn tìm phòng trống	50
6.3.3	Nhóm truy vấn gói dịch vụ	52
6.3.4	Nhóm truy vấn tài nguyên & tồn kho	54
6.3.5	Nhóm truy vấn giao dịch và doanh thu	55
6.3.6	Nhóm truy vấn cộng đồng	58
6.3.7	Nhóm truy vấn vận hành & bảo trì	60
6.3.8	Nhóm truy vấn nâng cao	62
6.4	Kiểm thử trigger	64
6.4.1	Phân tích một số trường hợp đáng chú ý	64
6.4.2	Hạn chế đã phát hiện qua kiểm thử	66

Chương 1

DẪN NHẬP ĐỀ TÀI

1.1 Bối cảnh thực tiễn

Trong khoảng một thập kỷ trở lại đây, nhiếp ảnh phim (film photography) đã có sự hồi sinh mạnh mẽ trong cộng đồng sáng tạo trẻ. Sự phát triển này đến từ giá trị nghệ thuật đặc trưng của chất liệu analog: dải động cao, hạt phim đặc trưng và quy trình sáng tác chậm rãi, có chủ đích.

Song song đó, nhu cầu về các nguồn lực phục vụ nhiếp ảnh phim cũng tăng rõ rệt, gồm ba nhóm chính: không gian sáng tạo (phòng tối, phòng chụp), thiết bị chuyên dụng (máy ảnh analog, ống kính, máy phóng) và vật tư tiêu hao (hóa chất tráng phim, giấy ảnh).

Tuy nhiên, thị trường cung cấp các nguồn lực này đang phân mảnh nghiêm trọng. Người chụp phim phải tìm kiếm qua nhiều kênh rời rạc như hội nhóm Facebook, Zalo hay giới thiệu cá nhân. Việc kiểm tra lịch trống, hỏi thông tin thiết bị và xác nhận đặt chỗ đều thực hiện thủ công. Ở phía nhà cung cấp, đa số vẫn quản lý bằng bảng tính hoặc sổ sách, dẫn đến hiệu quả vận hành thấp và khó mở rộng quy mô.

Sự thiếu vắng một nền tảng tích hợp kết nối cộng đồng nhiếp ảnh phim với các nhà cung cấp dịch vụ là khoảng trống rõ ràng cả về kỹ thuật lẫn thị trường. Đây chính là động lực để nhóm lựa chọn đề tài xây dựng cơ sở dữ liệu cho nền tảng số kết nối đa bên này.

1.2 Vấn đề đặt ra cho bài toán dữ liệu

Việc xây dựng nền tảng tích hợp đòi hỏi giải quyết đồng thời nhiều bài toán dữ liệu phức tạp:

- **Dữ liệu phân tán, không có chuẩn chung.** Thông tin về phòng tối, thiết bị và dịch vụ được quảng bá trên nhiều nền tảng với định dạng không thống nhất, gây khó khăn cho việc tìm kiếm và so sánh tự động.
- **Trùng lịch đặt chỗ.** Hiện tượng hai người cùng đặt một phòng trong cùng khung giờ là lỗi nghiêm trọng. Việc kiểm tra trùng lịch không thể chỉ dựa vào tăng ứng dụng mà cần

cơ chế cưỡng chế ở tầng cơ sở dữ liệu.

- **Vòng đời thiết bị và vật tư tiêu hao.** Thiết bị có lịch bảo trì và trạng thái khả dụng riêng; vật tư tiêu hao cần theo dõi tồn kho để tránh hết hàng giữa ca sử dụng.
- **Ràng buộc toàn vẹn đa tầng.** Một booking có thể liên quan đồng thời đến không gian, nhiều thiết bị, gói dịch vụ và chuyên gia hướng dẫn, đòi hỏi thiết kế ràng buộc chặt chẽ giữa nhiều bảng.
- **Dữ liệu đa hình.** Gói dịch vụ chứa nhiều loại item khác nhau; đánh giá có thể hướng đến nhiều loại đối tượng — đây là bài toán thiết kế đặc thù, không thể giải quyết bằng khóa ngoại đơn thuần.

1.3 Mục tiêu đề án

Thiết kế và triển khai một cơ sở dữ liệu quan hệ chuẩn hóa, đầy đủ ràng buộc toàn vẹn, phục vụ nền tảng kết nối cộng đồng nhiếp ảnh phim với các nhà cung cấp dịch vụ phòng tối, phòng chụp và tài nguyên sáng tạo.

- Mô hình hóa sáu luồng nghiệp vụ cốt lõi của nền tảng.
- Thiết kế ERD đầy đủ, mô tả chính xác thực thể, thuộc tính và mối quan hệ.
- Chuẩn hóa cơ sở dữ liệu đến dạng 3NF và BCNF, có phân tích phụ thuộc hàm.
- Triển khai trên MySQL 8.0.46 với đầy đủ ràng buộc, chỉ mục và trigger nghiệp vụ.
- Xây dựng dữ liệu mẫu và bộ truy vấn minh họa cho từng nhóm nghiệp vụ.
- Kiểm thử tính toàn vẹn dữ liệu thông qua các tình huống vi phạm ràng buộc.
- Đảm bảo cơ chế chống trùng lịch đặt chỗ được cưỡng chế ở tầng cơ sở dữ liệu.

1.4 Quy trình nghiệp vụ tổng quan

Hệ thống vận hành xoay quanh một vòng đời nghiệp vụ khép kín:

1. **Đăng ký và phân quyền.** Người dùng đăng ký với một trong các vai trò: nhiếp ảnh gia, nhà cung cấp, chuyên gia hoặc quản trị viên.
2. **Nhà cung cấp đăng tải tài nguyên.** Tạo hồ sơ không gian, thiết bị và vật tư tiêu hao với thông số và giá thuê.
3. **Tạo gói dịch vụ.** Kết hợp không gian, thiết bị và chuyên gia thành gói trọn gói cho từng loại hoạt động.
4. **Tìm kiếm và đặt chỗ.** Nhiếp ảnh gia chọn khung giờ và tài nguyên; hệ thống kiểm tra khả dụng và chống trùng lịch.
5. **Thanh toán và xác nhận.** Ghi nhận giao dịch, áp dụng khuyến mãi, cập nhật trạng thái booking.
6. **Check-in và sử dụng dịch vụ.** Khách check-in bằng mã QR; hệ thống ghi nhận thời gian vào thực tế.

7. **Check-out và tính phí.** Ghi nhận thời gian ra, tính chi phí phát sinh, cập nhật trạng thái COMPLETED.
8. **Đánh giá và phản hồi.** Khách để lại đánh giá hoặc gửi khiếu nại nếu có vấn đề.
9. **Bảo trì định kỳ.** Nhà cung cấp lên lịch bảo trì; hệ thống chặn đặt chỗ trùng với lịch bảo trì.

Chương 2

PHÂN TÍCH YÊU CẦU HỆ THỐNG

2.1 Yêu cầu chức năng theo tác nhân

Hệ thống phục vụ nền tảng kết nối cộng đồng nhiếp ảnh phim với dịch vụ phòng tối và phòng chụp, với 5 nhóm tác nhân chính. Vai trò của người dùng được lưu tại cột `users.role` (ENUM: `photographer`, `provider`, `expert`, `admin`); AI Assistant là tác nhân hệ thống, không phải tài khoản người dùng thông thường.

1. **Photographer (Nhiếp ảnh gia)** là người dùng phổ thông của nền tảng. Họ có thể tìm kiếm và đặt không gian như studio hoặc phòng tối, thuê thêm thiết bị đi kèm, đặt các gói dịch vụ trọn gói, và thực hiện thanh toán. Ngoài ra, họ còn có thể tham gia các buổi workshop, đăng bài chia sẻ, đánh giá dịch vụ đã sử dụng, quản lý bộ sưu tập ảnh cá nhân của mình, gửi khiếu nại khi cần, và trò chuyện với AI Assistant để được hỗ trợ.
2. **Provider (Nhà cung cấp)** là chủ sở hữu các không gian và thiết bị, được xác định trong hệ thống qua trường `users.role = 'provider'`. Vai trò của họ là khai báo và quản lý các không gian (`spaces`) cũng như thiết bị (`resources`) mà mình sở hữu, tạo ra các gói dịch vụ trọn gói (`service_packages`), thiết lập các chương trình khuyến mãi (`promotions`), lên lịch bảo trì cho không gian và thiết bị của mình, đồng thời theo dõi các booking (đơn đặt) đổ về từ phía khách hàng.
3. **Expert (Chuyên gia)** là người có chuyên môn trong lĩnh vực xử lý ảnh, được xác định qua `users.role = 'expert'`. Họ tổ chức các workshop để chia sẻ kỹ thuật, đăng các bài viết hoặc hướng dẫn chuyên sâu dưới dạng `guide` trong hệ thống posts, và có thể cung cấp dịch vụ tư vấn hoặc chỉnh sửa ảnh thông qua các gói dịch vụ.
4. **Admin (Quản trị viên)** là người quản trị toàn bộ hệ thống, được xác định qua `users.role = 'admin'`. Nhiệm vụ của Admin bao gồm quản lý danh mục dùng chung (`categories`), tạo và duyệt lịch bảo trì, xử lý các khiếu nại từ người dùng (`complaints`), giám sát các giao dịch thanh toán trong hệ thống, và quản lý tài khoản của toàn bộ người dùng.
5. **AI Assistant** là một tác nhân hệ thống, đóng vai trò hỗ trợ trực tuyến cho người dùng.

AI Assistant trả lời các câu hỏi, gợi ý không gian hoặc gói dịch vụ phù hợp với nhu cầu của người dùng thông qua các phiên chat, và toàn bộ lịch sử hội thoại được ghi nhận và lưu trữ tại bảng `chat_logs` để phục vụ việc theo dõi và cải thiện chất lượng hỗ trợ.

2.2 Sáu luồng nghiệp vụ cốt lõi

Dựa trên lược đồ cơ sở dữ liệu đã hiện thực (`Database_Schema.sql`, `nguoi1-nguoi5.sql`, `triggers.sql`), nghiệp vụ của hệ thống được chia thành 6 luồng chính như sau:

Luồng 1 — Đăng ký / Đăng nhập & Quản lý tài khoản

Tác nhân liên quan: Tất cả tác nhân

Bảng dữ liệu liên quan: `users`

Mô tả: Người dùng đăng ký và chọn vai trò (`photographer/provider/expert/admin`); mật khẩu được lưu dạng băm (`password_hash`); đăng nhập xác thực bằng JWT (theo core code Người 1); `provider/expert` có thể bổ sung thông tin doanh nghiệp (`business_name`, `address`, `description`).

Luồng 2 — Tìm kiếm & Đặt chỗ (Space/Resource)

Tác nhân liên quan: `Photographer`, `Provider`

Bảng dữ liệu liên quan: `spaces`, `resources`, `bookings`, `booking_resources`, `maintenance_schedule`, `favorites`

Mô tả: Người dùng tìm không gian theo loại (`DARKROOM/STUDIO`), xem chi tiết, chọn khung giờ và thiết bị đi kèm; hệ thống dùng trigger để chặn trùng lịch với booking khác hoặc với lịch bảo trì trước khi tạo `booking + booking_resources`; sinh mã QR (`qr_code_hash`) phục vụ check-in/check-out (`checkin_at`, `checkout_at`).

Luồng 3 — Đặt gói dịch vụ & Thanh toán

Tác nhân liên quan: `Photographer`, `Provider`

Bảng dữ liệu liên quan: `service_packages`, `package_items`, `promotions`, `bookings`, `transactions`

Mô tả: Người dùng có thể chọn một gói dịch vụ trọn gói (`space + resource`, `package_id`) thay vì đặt lẻ; áp mã khuyến mãi (`promotion_id`) để được giảm giá (`discount_amount`); thanh toán qua `VNPAY/MOMO/STRIPE/CASH`; `used_count` của khuyến mãi được trigger tự động cập nhật.

Luồng 4 — Quản lý bảo trì không gian & thiết bị

Tác nhân liên quan: Provider, Admin

Bảng dữ liệu liên quan: `maintenance_schedules`

Mô tả: Provider/Admin lên lịch bảo trì cho một không gian hoặc thiết bị cụ thể (`target_type`, `target_id`) với trạng thái `planned` → `in_progress` → `completed/cancelled`; hệ thống dùng lịch này để chặn các booking trùng thời gian bảo trì (Luồng 2).

Luồng 5 — Tương tác cộng đồng & Workshop

Tác nhân liên quan: Photographer, Expert

Bảng dữ liệu liên quan: `posts`, `reviews`, `photo_collections`, `photos`, `workshops`, `workshop_registrations`

Mô tả: Người dùng đăng bài chia sẻ/hướng dẫn, đánh giá không gian hoặc dịch vụ đã trải nghiệm, tạo bộ sưu tập ảnh cá nhân; Expert tổ chức workshop, người dùng đăng ký tham gia — số lượng đăng ký (`registered_count`) và giới hạn sức chứa được đồng bộ tự động bằng trigger.

Luồng 6 — Khiếu nại & Hỗ trợ

Tác nhân liên quan: Photographer, Admin, AI Assistant

Bảng dữ liệu liên quan: `complaints`, `chat_logs`, `notifications`

Mô tả: Người dùng gửi khiếu nại liên quan một booking cụ thể, Admin tiếp nhận và phản hồi (resolution); người dùng có thể trò chuyện với AI Assistant để được hỗ trợ nhanh; hệ thống gửi thông báo (notifications) cho các sự kiện quan trọng (xác nhận booking, thanh toán thành công, cảnh báo bảo trì...).

Chương 3

THIẾT KẾ MỨC QUAN NIỆM

3.1 Danh mục thực thể và ý nghĩa

Toàn hệ thống gồm 24 thực thể, chia theo 5 nhóm phụ trách:

Nhóm 1 – Người dùng

Thực thể	Ý nghĩa
users	Tài khoản duy nhất cho cả 4 vai trò: photographer, provider, expert, admin. Là gốc mọi khóa ngoại của hệ thống.

Nhóm 2 – Không gian & Đặt chỗ (Database_Schema)

Thực thể	Ý nghĩa
spaces	Phòng tối (darkroom) hoặc studio provider cung cấp, có giá theo giờ.
resources	Thiết bị/vật tư gắn với một space (máy ảnh, ống kính, hóa chất, giấy...)
bookings	Lượt đặt một space trong khung giờ cụ thể.
booking_resources	Chi tiết thiết bị được thuê kèm theo một booking (quan hệ N-N).

Nhóm 3 – Cộng đồng & Workshop

Thực thể	Ý nghĩa
reviews	Đánh giá/xếp hạng cho một đối tượng bất kì (studio, dịch vụ, workshop) theo mô hình polymorphic.
photo_collections	Bộ sưu tập ảnh cá nhân do người dùng tạo.
photos	Từng ảnh thuộc một bộ sưu tập.
posts	Bài viết/hướng dẫn chia sẻ trong cộng đồng (loại article hoặc guide).
workshops	Buổi workshop do một expert/provider tổ chức, có giới hạn số lượng người tham gia.
workshop_registrations	Lượt đăng kí tham gia workshop của người dùng.

Nhóm 4 – Gói dịch vụ, danh mục, giao dịch

Thực thể	Ý nghĩa
service_packages	Gói dịch vụ do provider đóng gói từ nhiều space/resource/chuyên gia.
package_items	Thành phần cụ thể (space/resource/user) thuộc một gói dịch vụ (polymorphic).
categories	Danh mục dùng chung, gắn nhãn cho space/resource/post/package.
space_categories, resource_categories, post_categories	Bảng trung gian gắn danh mục cho từng loại đối tượng.
promotions	Mã khuyến mãi do provider cấu hình, có thời hạn và số lần dùng tối đa.
transactions	Giao dịch thanh toán (booking hoặc workshop) qua các cổng VNPAY/MOMO/STRIPE/CASH.
complaints	Khiếu nại của khách hàng liên quan đến một booking.
chat_logs	Lịch sử hội thoại với trợ lý AI, lưu theo session.

Nhóm 5 – Bảo trì, yêu thích, thông báo

Thực thể	Ý nghĩa
maintenance_schedules	Lịch bảo trì cho space hoặc resource, dùng để chặn đặt chỗ trùng thời gian.

Thực thể	Ý nghĩa
favorites	Danh sách space được người dùng yêu thích.

3.2 Mô tả thuộc tính từng thực thể

3.2.1 USERS

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT, AUTO_INCREMENT	PK	Định danh duy nhất của người dùng
name	VARCHAR(100)	NOT NULL	Họ tên hiển thị
email	VARCHAR(100)	UNIQUE, NOT NULL, CHECK chk_users_email_format (email LIKE '%_@_%_%')	Email đăng nhập, đồng thời là định danh duy nhất
password_hash	VARCHAR(255)	NOT NULL	Mật khẩu đã băm (không lưu plain-text)
role	ENUM('photographer', 'provider', 'expert', 'admin')	NOT NULL	Phân quyền người dùng, quyết định tập chức năng được truy cập
phone	VARCHAR(20)	NULL	Số điện thoại liên hệ
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm tạo tài khoản
business_name	VARCHAR(150)	NULL (bổ sung sau bằng ALTER TABLE)	Tên doanh nghiệp/studio, chỉ có ý nghĩa với provider/expert
address	VARCHAR(255)	NULL (bổ sung sau bằng ALTER TABLE)	Địa chỉ liên hệ/kinh doanh
description	TEXT	NULL (bổ sung sau bằng ALTER TABLE)	Mô tả giới thiệu, chỉ có ý nghĩa với provider/expert

3.2.2 SPACES

Cột	Kiểu dữ liệu	Ràng buộc	Diễn giải
id	INT, AUTO_INCREMENT	PRIMARY KEY	Định danh không gian
provider_id	INT	NOT NULL, FK → users(id) ON DELETE CASCADE	Provider sở hữu không gian
name	VARCHAR(255)	NOT NULL	Tên không gian
type	ENUM('DARKROOM', 'STUDIO')	NOT NULL	Loại không gian
description	TEXT		Mô tả chi tiết
capacity	INT	NOT NULL, CHECK (capacity > 0)	Sức chứa tối đa
hourly_rate	DECIMAL(10,2)	NOT NULL, CHECK (hourly_rate >= 0)	Giá thuê theo giờ
operating_hours	JSON		Giờ hoạt động, vd. {"open": "08:00", "close": "22:00"}
specifications	JSON		Thông số kỹ thuật (ánh sáng, phong nền, hoá chất...)
status	ENUM('AVAILABLE', 'MAINTENANCE', 'INACTIVE')	NOT NULL	Trạng thái không gian
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm tạo

Chỉ mục (Index): idx_spaces_provider (provider_id, status); idx_spaces_type (type, status).

3.2.3 RESOURCES

Cột	Kiểu dữ liệu	Ràng buộc	Diễn giải
id	INT, AUTO_INCREMENT	PRIMARY KEY	Định danh thiết bị
space_id	INT	NOT NULL, FK → spaces(id) ON DELETE CASCADE	Không gian chứa thiết bị
name	VARCHAR(255)	NOT NULL	Tên thiết bị
category	ENUM('CAMERA', 'EYEGLASS', 'LARGER', 'SCANNER', 'LIGHTING', 'CHEMICAL')	NOT NULL	Loại thiết bị
serial_number	VARCHAR(100)		Số serial

Cột	Kiểu dữ liệu	Ràng buộc	Diễn giải
rental_price	DECIMAL(10,2)	DEFAULT 0.00, CHECK (rental_price >= 0)	Giá thuê thiết bị
stock_quantity	INT	NULL (bổ sung sau)	Số lượng tồn kho
unit	VARCHAR(20)	NULL (bổ sung sau)	Đơn vị tính (set, unit, cuộn. . .)
status	ENUM('AVAILABLE', 'IN USE', 'MAINTENANCE', 'DAMAGED')	DEFAULT 'AVAILABLE'	Trạng thái thiết bị

Chỉ mục (Index): idx_resources_space (space_id, status); idx_resources_category (category, status).

3.2.4 MAINTENANCE_SCHEDULES

Cột	Kiểu dữ liệu	Ràng buộc	Diễn giải
id	INT, AUTO_INCREMENT	PRIMARY KEY	Định danh lịch bảo trì
target_type	ENUM('space', 'resource')	NOT NULL	Loại đối tượng được bảo trì
target_id	INT	NOT NULL (FK đa hình, không ràng buộc vật lý)	id của space hoặc resource tương ứng
scheduled_start	DATETIME	NOT NULL	Thời điểm bắt đầu bảo trì
scheduled_end	DATETIME	NOT NULL, CHECK (scheduled_end > scheduled_start)	Thời điểm kết thúc bảo trì
reason	VARCHAR(255)		Lý do bảo trì
status	ENUM('planned', 'in progress', 'completed', 'cancelled')	DEFAULT 'planned'	Trạng thái bảo trì
created_by	INT	NOT NULL, FK → users(id) ON DELETE CASCADE	Người tạo lịch bảo trì
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm tạo bản ghi

Chỉ mục (Index): idx_maint_target_time (target_type, target_id, scheduled_start, scheduled_end, status).

3.2.5 BOOKINGS

Cột	Kiểu dữ liệu	Ràng buộc	Diễn giải
id	INT, AUTO_INCREMENT	PRIMARY KEY	Định danh booking
user_id	INT	NOT NULL, FK → users(id) ON DELETE CASCADE	Người đặt chỗ
space_id	INT	NOT NULL, FK → spaces(id) ON DELETE CASCADE	Không gian được đặt
package_id	INT	NULL, FK → service_packages(id) ON DELETE SET NULL	Gói dịch vụ áp dụng (nếu có)
start_time	DATETIME	NOT NULL	Thời điểm bắt đầu
end_time	DATETIME	NOT NULL, CHECK (end_time > start_time)	Thời điểm kết thúc
total_price	DECIMAL(10,2)	NOT NULL, CHECK (total_price >= 0)	Tổng giá trị booking
status	ENUM('PENDING', 'CONFIRMED', 'CHECKED_IN', 'COMPLETED', 'CANCELLED')	NOT NULL, DEFAULT 'PENDING'	Trạng thái booking
qr_code_hash	VARCHAR(255)	UNIQUE (uq_bookings_qr)	Mã QR check-in (cho phép NULL)
checkin_at	DATETIME	NULL	Thời điểm check-in thực tế
checkout_at	DATETIME	NULL, CHECK (checkout_at >= checkin_at hoặc một trong hai NULL)	Thời điểm check-out thực tế
promotion_id	INT	NULL, FK → promotions(id) ON DELETE SET NULL	Mã khuyến mãi áp dụng (nếu có)
discount_amount	DECIMAL(10,2)	DEFAULT 0, CHECK (discount_amount >= 0)	Số tiền được giảm

Cột	Kiểu dữ liệu	Ràng buộc	Diễn giải
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm tạo booking

Chỉ mục (Index): idx_bookings_space_time (space_id, start_time, end_time, status);
idx_bookings_user (user_id, status, start_time); uq_bookings_qr UNIQUE (qr_code_hash).

3.2.6 BOOKING_RESOURCES

Cột	Kiểu dữ liệu	Ràng buộc	Diễn giải
booking_id	INT	PK (phức hợp), FK → bookings(id) ON DELETE CASCADE	Booking liên quan
resource_id	INT	PK (phức hợp), FK → resources(id) ON DELETE CASCADE	Thiết bị được thuê
quantity	INT	DEFAULT 1, CHECK (quantity > 0)	Số lượng thiết bị

Chỉ mục (Index): Khóa chính phức hợp (booking_id, resource_id) đồng thời đóng vai trò chỉ mục truy vấn.

3.2.7 REVIEWS

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT, AUTO_INCREMENT	PK	Định danh đánh giá
user_id	INT	NOT NULL, FK → users.id	Người viết đánh giá
target_type	VARCHAR(50)	NOT NULL	Loại đối tượng được đánh giá: 'studio' 'service' 'workshop' (khóa polymorphic, không FK thật)
target_id	INT	NOT NULL	ID của đối tượng tương ứng với target_type

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
rating	TINYINT	NOT NULL, CHECK 1-5	Điểm đánh giá
comment	TEXT	NULL	Nội dung nhận xét
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm đánh giá

3.2.8 PHOTO_COLLECTIONS

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT, AUTO_INCREMENT	PK	Định danh bộ sưu tập
user_id	INT	NOT NULL, FK → users.id, ON DELETE CASCADE	Chủ sở hữu bộ sưu tập
title	VARCHAR(255)	NOT NULL	Tên bộ sưu tập
description	TEXT	NULL	Mô tả
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm tạo

3.2.9 PHOTOS

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT, AUTO_INCREMENT	PK	Định danh ảnh
collection_id	INT	NOT NULL, FK → photo_collections.id, ON DELETE CASCADE	Bộ sưu tập chứa ảnh
image_url	VARCHAR(500)	NOT NULL	Đường dẫn ảnh
caption	VARCHAR(500)	NULL	Chú thích ảnh
uploaded_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm tải lên

3.2.10 POSTS

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT, AUTO_INCREMENT	PK	Định danh bài viết
author_id	INT	NOT NULL, FK → users.id, ON DELETE CASCADE	Tác giả
title	VARCHAR(255)	NOT NULL	Tiêu đề
content	TEXT	NOT NULL	Nội dung
type	ENUM('article','guide')	DEFAULT 'article'	Loại bài viết
tags	VARCHAR(255)	NULL	Nhãn/thẻ, phân tách bằng dấu phẩy
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm đăng

3.2.11 WORKSHOPS

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT, AUTO_INCREMENT	PK	Định danh workshop
organizer_id	INT	NOT NULL, FK → users.id, ON DELETE CASCADE	Người tổ chức
title	VARCHAR(255)	NOT NULL	Tên workshop
description	TEXT	NULL	Mô tả nội dung
date	DATETIME	NOT NULL	Thời gian diễn ra
location	VARCHAR(255)	NULL	Địa điểm
capacity	INT	NOT NULL, CHECK > 0	Sức chứa tối đa
registered_count	INT	DEFAULT 0, CHECK ≥ 0	Số người đã đăng ký (đồng bộ tự động bằng trigger, mục 5.5)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm tạo

3.2.12 WORKSHOP_REGISTRATION

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT, AUTO_INCREMENT	PK	Định danh lượt đăng ký
workshop_id	INT	NOT NULL, FK → workshops.id, ON DELETE CASCADE	Workshop được đăng ký
user_id	INT	NOT NULL, FK → users.id, ON DELETE CASCADE	Người đăng ký
status	ENUM('PENDING', 'PAID', 'CANCELLED')	DEFAULT 'PENDING'	Trạng thái đăng ký
payment_id	INT	NULL, FK → transactions.id, ON DELETE SET NULL (thêm sau ở nguoi3.sql)	Giao dịch thanh toán liên kết
registered_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm đăng ký

Ràng buộc bổ sung: UNIQUE(workshop_id, user_id) — một người chỉ đăng ký một lần cho mỗi workshop.

3.2.13 SERVICE_PACKAGES

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT AUTO_INCREMENT	PK	Mã định danh duy nhất của gói dịch vụ
provider_id	INT	FK → users(id), NOT NULL	Mã nhà cung cấp sở hữu gói dịch vụ
name	VARCHAR(150)	NOT NULL	Tên gói dịch vụ hiển thị cho khách hàng
description	TEXT	NULL	Mô tả chi tiết nội dung gói
price	DECIMAL(10,2)	NOT NULL, CHECK price ≥ 0	Giá trọn gói của dịch vụ
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm khởi tạo gói

3.2.14 PACKAGE_ITEMS

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
package_id	INT	Mã gói dịch vụ chứa thành phần này	PK (thành phần), FK → service_packages(id)
item_type	ENUM('space', 'resource')	Loại thành phần được đưa vào gói	PK (thành phần), NOT NULL
item_id	INT	Mã của không gian / thiết bị / chuyên gia tương ứng	PK (thành phần), NOT NULL
quantity	INT	Số lượng thành phần trong gói	DEFAULT 1, CHECK quantity > 0

3.2.15 CATEGORIES

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
id	INT AUTO_INCREMENT	Mã định danh duy nhất của danh mục	PK
name	VARCHAR(100)	Tên danh mục, ví dụ: Analog, Chân dung, Sản phẩm	NOT NULL
applies_to	ENUM('space', 'resource')	Nhiệm vụ áp dụng danh mục áp dụng	NOT NULL
created_at	TIMESTAMP	Thời điểm tạo danh mục	DEFAULT CURRENT_TIMESTAMP
(name, applies_to)	—	Không cho phép trùng tên danh mục trong cùng một nhóm đối tượng	UNIQUE

3.2.16 SPACE_CATEGORIES

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
space_id	INT	Mã không gian được gán danh mục	PK (thành phần), FK → spaces(id)

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
category_id	INT	Mã danh mục được gắn	PK (thành phần), FK → categories(id)

3.2.17 RESOURCE_CATEGORIES

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
resource_id	INT	Mã thiết bị được gắn danh mục	PK (thành phần), FK → resources(id)
category_id	INT	Mã danh mục được gắn	PK (thành phần), FK → categories(id)

3.2.18 POST_CATEGORIES

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
post_id	INT	Mã bài viết được gắn danh mục	PK (thành phần), FK → posts(id)
category_id	INT	Mã danh mục được gắn	PK (thành phần), FK → categories(id)

3.2.19 PROMOTIONS

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
id	INT AUTO_INCREMENT	Mã định danh duy nhất của chương trình khuyến mãi	PK
provider_id	INT	Mã nhà cung cấp phát hành mã giảm giá	FK → users(id), NOT NULL
code	VARCHAR(50)	Mã khuyến mãi khách hàng nhập khi đặt chỗ	NOT NULL, UNIQUE
description	VARCHAR(255)	Mô tả ngắn về chương trình	NULL

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
discount_percent	TINYINT	Phần trăm giảm giá áp dụng	NOT NULL, CHECK BETWEEN 1 AND 100
start_at	DATETIME	Thời điểm bắt đầu hiệu lực	NOT NULL
end_at	DATETIME	Thời điểm kết thúc hiệu lực	NOT NULL, CHECK end_at > start_at
max_uses	INT	Số lượt sử dụng tối đa (NULL là không giới hạn)	DEFAULT NULL
used_count	INT	Số lượt đã sử dụng thực tế	DEFAULT 0, CHECK used_count >= 0
active	BOOLEAN	Trạng thái bật/tắt chương trình	DEFAULT TRUE
created_at	TIMESTAMP	Thời điểm tạo chương trình	DEFAULT CURRENT_TIMESTAMP

3.2.20 TRANSACTIONS

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
id	INT	Mã định danh duy nhất của giao dịch	PK
booking_id	INT	Mã đặt chỗ liên quan (nếu có)	FK → bookings(id), NULL
user_id	INT	Mã người dùng thực hiện thanh toán	FK → users(id), NOT NULL
amount	DECIMAL(10,2)	Số tiền của giao dịch	NOT NULL, CHECK amount > 0
method	ENUM('VNPAY', 'MOMO', 'STRIPE', 'CASH')	Phương thức thanh toán được sử dụng	NOT NULL
status	ENUM('PENDING', 'SUCCESS', 'FAILED', 'REFUNDED')	Trạng thái của giao dịch	'PENDING'
external_ref	VARCHAR(255)	Mã tham chiếu do cổng thanh toán trả về	NULL
note	VARCHAR(255)	Ghi chú bổ sung cho giao dịch	NULL

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
created_at	TIMESTAMP	Thời điểm khởi tạo giao dịch	DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP	Thời điểm cập nhật gần nhất	ON UPDATE CURRENT_TIMESTAMP
(method, external_ref)	—	Chống ghi trùng kết quả callback từ cùng một cổng thanh toán	UNIQUE

3.2.21 COMPLAINTS

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
id	INT AUTO_INCREMENT	Mã định danh duy nhất của khiếu nại	PK
booking_id	INT	Mã đặt chỗ bị khiếu nại	FK → bookings(id), NOT NULL
user_id	INT	Mã người dùng gửi khiếu nại	FK → users(id), NOT NULL
content	TEXT	Nội dung khiếu nại	NOT NULL
status	ENUM('OPEN', 'IN_PROGRESS', 'RESOLVED', 'REFLECTED', 'CLOSED')	Trạng thái của khiếu nại	DEFAULT 'OPEN'
resolution	TEXT	Nội dung phản hồi / hướng giải quyết	NULL
created_at	TIMESTAMP	Thời điểm gửi khiếu nại	DEFAULT CURRENT_TIMESTAMP
resolved_at	DATETIME	Thời điểm khiếu nại được xử lý xong	NULL

3.2.22 CHAT_LOGS

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
id	INT AUTO_INCREMENT	Mã định danh duy nhất của một lượt hội thoại	PK
user_id	INT	Mã người dùng tham gia hội thoại	FK → users(id), NOT NULL

Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
session_id	VARCHAR(100)	Mã phiên hội thoại	NOT NULL
role	ENUM('user','assistant','system')	Vai trò của bên phát ngôn trong lượt hội thoại	NOT NULL
message	TEXT	Nội dung thông điệp	NOT NULL
created_at	TIMESTAMP	Thời điểm ghi nhận thông điệp	DEFAULT CURRENT_TIMESTAMP

3.2.23 FAVORITES

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
user_id	INT	PK (thành phần), FK → users(id), NOT NULL, ON DELETE CASCADE	Người dùng thực hiện đánh dấu yêu thích
space_id	INT	PK (thành phần), FK → spaces(id), NOT NULL, ON DELETE CASCADE	Phòng được đánh dấu yêu thích
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm người dùng thêm yêu thích, dùng để sắp xếp danh sách theo thứ tự mới nhất

3.2.24 NOTIFICATIONS

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	PK, AUTO_INCREMENT	Định danh thông báo do hệ thống sinh
user_id	INT	FK → users(id), NOT NULL, ON DELETE CASCADE	Người nhận thông báo
title	VARCHAR(255)	NOT NULL	Tiêu đề ngắn hiển thị trên danh sách thông báo

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
message	TEXT	NOT NULL	Nội dung đầy đủ của thông báo
type	ENUM('booking', 'payment', 'complaint', 'review', 'photo_collection', 'workshop', 'maintenance', 'system')	DEFAULT 'booking'	Phân loại thông báo theo phân hệ phát sinh, dùng để lọc và gán biểu tượng
is_read	BOOLEAN	DEFAULT FALSE	Trạng thái đã đọc; nguồn dữ liệu cho chỉ báo số thông báo chưa đọc
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm phát sinh thông báo

3.3 Ma trận quan hệ giữa các thực thể

Thực thể A	Quan hệ	Thực thể B	Bậc
users	sở hữu	spaces	1-N
users	sở hữu	service_packages	1-N
users	sở hữu	promotions	1-N
users	thực hiện	bookings	1-N
users	thực hiện	transactions	1-N
users	gửi	complaints	1-N
users	viết	reviews	1-N
users	tạo	photo_collections	1-N
users	viết	posts	1-N
users	tổ chức	workshops	1-N
users	đăng ký	workshop_registrations	1-N
users	đánh dấu	favorites	1-N
users	nhận	notifications	1-N
users	phát sinh	chat_logs	1-N
users	tạo	maintenance_schedules (created_by)	1-N
spaces	chứa	resources	1-N
spaces	được đặt qua	bookings	1-N
spaces	được yêu thích qua	favorites	1-N
spaces	gắn danh mục qua	space_categories	N-N (với categories)

Thực thể A	Quan hệ	Thực thể B	Bậc
bookings	gồm	booking_resources	1-N
resources	được thuê qua	booking_resources	1-N
resources	gắn danh mục qua	resource_categories	N-N (với categories)
bookings	thuộc	service_packages (package_id)	N-1 (0 hoặc 1)
bookings	áp dụng	promotions (promotion_id)	N-1 (0 hoặc 1)
bookings	có	transactions	1-N
bookings	bị khiếu nại qua	complaints	1-N
service_packages	gồm	package_items	1-N
			(polymorphic → spaces/resources/users)
categories	gắn cho	space_categories / resource_categories / post_categories	N-N
posts	gắn danh mục qua	post_categories	N-N (với categories)
photo_collections	chứa	photos	1-N
workshops	nhận	workshop_registrations	1-N

3.4 Quy tắc nghiệp vụ (Business Rules)

Nhóm A — Người dùng & phân quyền

- **BR1:** Mỗi email chỉ được đăng ký một tài khoản duy nhất (UNIQUE(email)).
- **BR2:** Email phải đúng định dạng cơ bản `__@__.` (chk_users_email_format).
- **BR3:** Mỗi người dùng có duy nhất một vai trò trong 4 vai trò cố định (ENUM role).

Nhóm B — Không gian & Đặt chỗ

- **BR4:** Thời gian kết thúc booking phải sau thời gian bắt đầu (chk_bookings_time).
- **BR5:** Một space không được có hai booking đang hoạt động (PENDING/CONFIRMED/CHECKED) trùng khung giờ (trg_bookings_overlap_bi/bu).
- **BR6:** Không được đặt space trong thời gian space đó đang bảo trì (planned/in_progress).
- **BR7:** Một resource không được xuất hiện trong hai booking đang hoạt động trùng khung giờ (trg_bookings_resources_overlap_bi), và cũng không được đặt khi resource đang bảo trì.
- **BR8:** Thời gian check-out phải sau hoặc bằng thời gian check-in (chk_bookings_checkout_after_checkin).

- **BR9:** Sức chứa (capacity) và giá thuê theo giờ (hourly_rate) của space phải là số dương/không âm.

Nhóm C — Gói dịch vụ & Khuyến mãi

- **BR10:** Giá gói dịch vụ không âm.
- **BR11:** % giảm giá của khuyến mãi trong khoảng 1–100 (chk_promo_percent).
- **BR12:** Thời gian kết thúc khuyến mãi phải sau thời gian bắt đầu.
- **BR13:** Số lần khuyến mãi được sử dụng (used_count) được đồng bộ tự động khi booking gần/hủy mã (trg_bookings_promo_ai/au).

Nhóm D — Giao dịch & Khiếu nại

- **BR14:** Số tiền giao dịch phải lớn hơn 0.
- **BR15:** Cặp (phương thức thanh toán, mã tham chiếu ngoài) phải duy nhất (uq_tx_external_ref) để tránh xử lý trùng một giao dịch.
- **BR16:** Một khiếu nại luôn phải gắn với một booking cụ thể đã tồn tại.

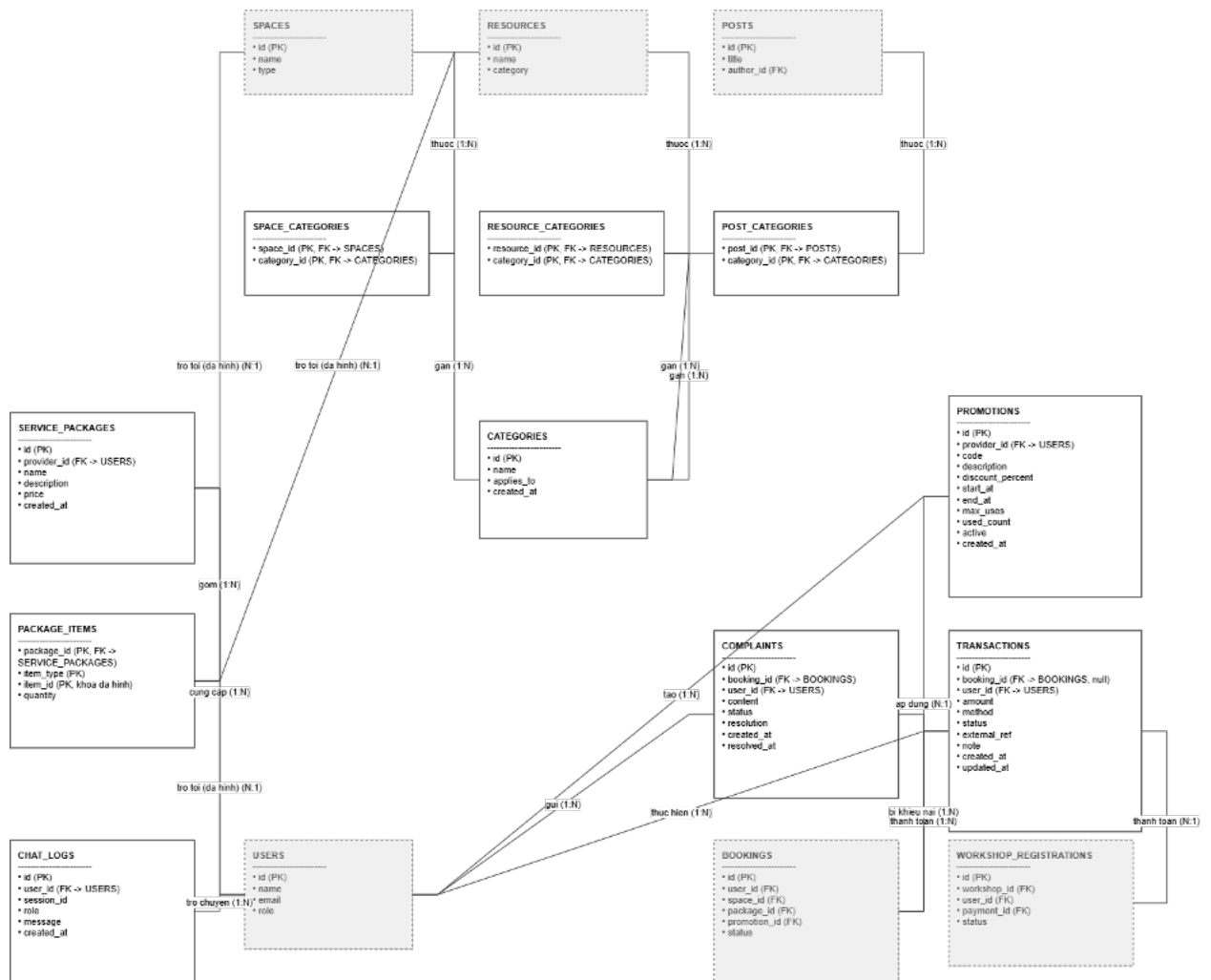
Nhóm E — Cộng đồng & Workshop

- **BR17:** Điểm đánh giá (rating) chỉ nhận giá trị từ 1 đến 5.
- **BR18:** Sức chứa workshop (capacity) phải lớn hơn 0; số người đã đăng ký (registered_count) không được âm.
- **BR19:** Một người dùng chỉ được đăng ký một lần cho mỗi workshop (UNIQUE(workshop_id, user_id)).
- **BR20:** Không được đăng ký workshop khi đã đủ số lượng ($\text{registered_count} \geq \text{capacity}$) — kiểm tra ở trg_workshop_reg_capacity.
- **BR21:** Không được đăng ký workshop đã diễn ra ($\text{date} < \text{NOW}()$).
- **BR22:** registered_count của workshop phải luôn phản ánh đúng số đăng ký đang hợp lệ (không phải CANCELLED) — được đồng bộ tự động qua 4 trigger ở mục 5.5, không cho phép ứng dụng tự cập nhật cột này trực tiếp.

Nhóm F — Toàn vẹn tham chiếu chung

- **BR23:** Xóa một người dùng sẽ cascade xóa toàn bộ dữ liệu phụ thuộc trực tiếp (space, booking, review, post, workshop do họ tạo...) — áp dụng ON DELETE CASCADE cho phần lớn quan hệ với users.
- **BR24:** Xóa promotion hoặc booking liên kết với transaction sẽ set về NULL (ON DELETE SET NULL) để giữ lại lịch sử giao dịch thay vì xóa cứng.

3.5 Sơ đồ ERD



Hình 3.1: Sơ đồ ERD của hệ thống

Chương 4

THIẾT KẾ MỨC LOGIC

4.1 Lược đồ quan hệ

- **USERS** (**id**, **name**, **email**, **password_hash**, **role**, **phone**, **created_at**, **business_name**, **address**, **description**)
PK: { **id** }
FK: không có — USERS là bảng gốc (root entity) của toàn hệ thống, không tham chiếu ra bất kỳ bảng nào khác. Ngược lại, USERS là đích tham chiếu của 15 khóa ngoại từ 15 bảng khác trong hệ thống.
- **SPACES** (**id**, **provider_id***, **name**, **type**, **description**, **capacity**, **hourly_rate**, **operating_hours**, **specifications**, **status**, **created_at**)
FK: **provider_id** → USERS(**id**)
- **RESOURCES** (**id**, **space_id***, **name**, **category**, **serial_number**, **rental_price**, **stock_quantity**, **unit**, **status**)
FK: **space_id** → SPACES(**id**)
- **BOOKINGS** (**id**, **user_id**, **space_id**, **package_id**, **start_time**, **end_time**, **total_price**, **status**, **qr_code_hash**, **checkin_at**, **checkout_at**, **promotion_id**, **discount_amount**, **created_at**)
FK: **user_id** → USERS(**id**)
FK: **space_id** → SPACES(**id**)
FK: **package_id** → SERVICE_PACKAGES(**id**) (thêm sau ở nguoi2.sql)
FK: **promotion_id** → PROMOTIONS(**id**) (thêm sau ở nguoi3.sql)
- **BOOKING_RESOURCES** (**booking_id**, **resource_id**, **quantity**)
PK: { **booking_id**, **resource_id** }
FK: **booking_id** → BOOKINGS(**id**)
FK: **resource_id** → RESOURCES(**id**)
- **MAINTENANCE_SCHEDULES** (**id**, **target_type**, **target_id**, **scheduled_start**, **scheduled_end**, **reason**, **status**, **created_by***, **created_at**)

FK: created_by → USERS(id)

target_id là khóa ngoại đa hình (polymorphic FK) — không có ràng buộc FOREIGN KEY vật lý.

- **REVIEWS** (id, user_id*, target_type, target_id, rating, comment, created_at)
FK: user_id → USERS(id)
- **PHOTO_COLLECTIONS** (id, user_id*, title, description, created_at)
FK: user_id → USERS(id)
- **PHOTOS** (id, collection_id*, image_url, caption, uploaded_at)
FK: collection_id → PHOTO_COLLECTIONS(id)
- **POSTS** (id, author_id*, title, content, type, tags, created_at)
FK: author_id → USERS(id)
- **WORKSHOPS** (id, organizer_id*, title, description, date, location, capacity, registered_count, created_at)
FK: organizer_id → USERS(id)
- **WORKSHOP_REGISTRATIONS** (id, workshop_id, user_id, status, payment_id*, registered_at)
FK: workshop_id → WORKSHOPS(id)
FK: user_id → USERS(id)
FK: payment_id → TRANSACTIONS(id) - thêm sau ở nguoi3.sql
UNIQUE (workshop_id, user_id)
- **SERVICE_PACKAGES** (id, provider_id*, name, description, price, created_at)
FK: provider_id → USERS(id)
- **PACKAGE_ITEMS** (package_id*, item_type, item_id, quantity)
PK: { package_id, item_type, item_id }
FK: package_id → SERVICE_PACKAGES(id)
item_id là khóa đa hình - không có FK vật lý.
- **CATEGORIES** (id, name, applies_to, created_at)
Không có FK.
- **SPACE_CATEGORIES** (space_id, category_id)
PK: { space_id, category_id }
FK: space_id → SPACES(id)
FK: category_id → CATEGORIES(id)
- **RESOURCE_CATEGORIES** (resource_id, category_id)
PK: { resource_id, category_id }
FK: resource_id → RESOURCES(id)
FK: category_id → CATEGORIES(id)
- **POST_CATEGORIES** (post_id, category_id)

- PK: { post_id, category_id }
- FK: post_id → POSTS(id)
- FK: category_id → CATEGORIES(id)
- **PROMOTIONS** (id, provider_id*, code, description, discount_percent, start_at, end_at, max_uses, used_count, active, created_at)
FK: provider_id → USERS(id)
 - **TRANSACTIONS** (id, booking_id, user_id, amount, method, status, external_ref, note, created_at, updated_at)
FK: booking_id → BOOKINGS(id)
FK: user_id → USERS(id)
 - **COMPLAINTS** (id, booking_id, user_id, content, status, resolution, created_at, resolved_at)
FK: booking_id → BOOKINGS(id)
FK: user_id → USERS(id)
 - **CHAT_LOGS** (id, user_id*, session_id, role, message, created_at)
FK: user_id → USERS(id)
 - **FAVORITES** (user_id, space_id, created_at)
PK: { user_id, space_id }
FK1: user_id → USERS(id) ON DELETE CASCADE
FK2: space_id → SPACES(id) ON DELETE CASCADE
 - **NOTIFICATIONS** (id, user_id*, title, message, type, is_read, created_at)
PK: { id }
FK1: user_id → USERS(id) ON DELETE CASCADE

4.2 Chuẩn hóa dữ liệu

4.2.1 Kiểm tra 1NF

Điều kiện 1NF: mỗi thuộc tính mang giá trị nguyên tố (atomic), không tồn tại nhóm lặp hay danh sách nhúng trong một ô dữ liệu, và mỗi bảng có khóa chính xác định duy nhất mỗi dòng.

Rà soát toàn bộ 24 bảng cho thấy mọi cột đều mang giá trị đơn, không có cột nào lưu danh sách giá trị phân tách bằng dấu phẩy hay mảng nhúng kiểu bảng-trong-bảng; mọi bảng đều có khóa chính (surrogate key id AUTO_INCREMENT hoặc khóa chính phức hợp) xác định duy nhất một dòng dữ liệu, ví dụ mỗi dòng của bookings mô tả đúng một lượt đặt chỗ, mỗi dòng của package_items mô tả đúng một thành phần trong một gói dịch vụ.

Ngoại lệ có chủ đích duy nhất là hai cột `spaces.operating_hours` và `spaces.specifications` dùng kiểu JSON để lưu cấu trúc linh hoạt (giờ mở/đóng cửa; thông số ánh sáng, phong nền, hóa chất... khác nhau theo từng loại không gian). Đây là đánh đổi có chủ đích: chấp nhận vi

phạm tính "nguyên tố tuyệt đối" của 1NF trên hai cột này để tránh phải tách thêm các bảng thuộc tính (space_operating_hours, space_specifications...) cho những trường có cấu trúc thay đổi theo loại không gian/dịch vụ, giúp schema dễ mở rộng khi phát sinh loại không gian mới mà không cần sửa cấu trúc bảng. Ngoài hai cột này, toàn bộ 24 bảng đạt 1NF.

4.2.2 Kiểm tra 2NF

Bảng	Khóa chính phức hợp	Thuộc tính không khóa	Kiểm tra phụ thuộc đầy đủ
booking_resources	(booking_id, resource_id)	quantity	Phụ thuộc cả cặp khóa, không suy ra từ riêng lẻ → đạt 2NF
package_items	(package_id, item_type, item_id)	quantity	Phụ thuộc toàn bộ 3 cột → đạt 2NF
favorites	(user_id, space_id)	created_at	Phụ thuộc cả hai vế → đạt 2NF
space_categories	(space_id, category_id)	—	Không có thuộc tính không khóa → đạt 2NF hiển nhiên
resource_categories	(resource_id, category_id)	—	Tương tự space_categories → thỏa 2NF hiển nhiên.
post_categories	(post_id, category_id)	—	Tương tự space_categories → thỏa 2NF hiển nhiên.

4.2.3 Kiểm tra 3NF

Cột	Bảng	Nguy cơ phụ thuộc bắc cầu	Lý do được chấp nhận / cơ chế đảm bảo toàn vẹn
total_price, discount_amount	bookings	Có thể nhầm là suy ra từ hourly_rate/rental_price/distance, không phải bắc cầu nội ở bảng khác nhân thời lượng	Là phụ thuộc liên bảng (cross-table), không phải bắc cầu nội bộ trong bookings nên không vi phạm 3NF. Giá trị được "đóng băng" có chủ đích tại thời điểm giao dịch để bảo toàn lịch sử — đơn giá/khuyến mãi có thể đổi sau, nhưng total_price của booking đã hoàn tất không được thay đổi theo

Cột	Bảng	Nguyên cơ phụ thuộc bắt cầu	Lý do được chấp nhận / cơ chế đảm bảo toàn vẹn
registered_count	workshops	Là COUNT từ workshop_registrations	Trigger trg_workshop_reg_capacity/ai/ad/au tự động cập nhật sau mỗi INSERT/DELETE/UPDATE status
used_count	promotions	Là COUNT bookings đã dùng promotion_id	Trigger trg_bookings_promo_ai/au tự động cập nhật, kể cả khi đổi mã khuyến mãi
business_name, address, description	users	Chỉ có ý nghĩa khi role = 'provider'/'expert'	Không phải phụ thuộc hàm thực sự, chỉ là NULL có điều kiện theo vai trò — ràng buộc nghiệp vụ, không vi phạm 3NF. Giữ NULL để tránh JOIN thêm cho xác thực tần suất cao

4.2.4 Phân tích phụ thuộc hàm và BCNF

Bảng	Khóa chính (PK)	Ứng viên khóa phụ (UNIQUE)	Đánh giá BCNF
users	id	email (UNIQUE NOT NULL)	Cả hai → tất cả thuộc tính → đạt BCNF
categories	id	(name, applies_to) — UNIQUE	Cả hai là siêu khóa → đạt BCNF
promotions	id	code (UNIQUE NOT NULL)	Cả hai là siêu khóa → đạt BCNF
transactions	id	(method, external_ref) — UNIQUE uq_tx_external_ref	external_ref có thể NULL (giao dịch CASH) nên không phải ứng viên khóa bắt buộc; id vẫn là determinant duy nhất → đạt BCNF
bookings	id	qr_code_hash (UNIQUE, cho phép NULL)	NULL đến khi generate QR → không phải ứng viên khóa thực sự → đạt BCNF qua id
workshop_registrations	id	(workshop_id, user_id) — UNIQUE unique_registration	Cả hai là siêu khóa → đạt BCNF

Chương 5

TRIỂN KHAI CSDL VẬT LÝ

5.1 Môi trường và công cụ

Hạng mục	Cấu hình sử dụng
Hệ quản trị	MySQL 8.0 (Community Edition)
Storage engine	InnoDB
Character set / Collation	utf8mb4 / utf8mb4_unicode_ci
Tên cơ sở dữ liệu	film_photo_booking_platform
Công cụ quản trị	MySQL Workbench, DBeaver
Công cụ sao lưu	mysqldump

5.2 Script DDL — Tạo bảng

Toàn bộ mã tạo bảng của phần phụ trách nằm trong tệp `nguoi3.sql`, được chia thành sáu phần theo cụm chức năng: gói dịch vụ, danh mục, khuyến mãi, giao dịch, khiếu nại và nhật ký trợ lý AI.

5.2.1 Phần A — Gói dịch vụ

Khóa chính ghép (`package_id`, `item_type`, `item_id`) bảo đảm một thành phần không bị khai báo trùng hai lần trong cùng một gói. Hành vi ON DELETE CASCADE được chọn vì thành phần gói không có ý nghĩa độc lập: khi gói dịch vụ bị xóa, toàn bộ dòng chi tiết cũng phải biến mất.

5.2.2 Phần B — Danh mục và ba bảng trung gian

```
1 CREATE TABLE IF NOT EXISTS categories (  
2     id INT AUTO_INCREMENT PRIMARY KEY,
```

```

3     name VARCHAR(100) NOT NULL,
4     applies_to ENUM('space', 'resource', 'post', 'package') NOT NULL,
5     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
6     UNIQUE KEY uq_categories_name_applies (name, applies_to)
7 );
8
9 CREATE TABLE IF NOT EXISTS space_categories (
10     space_id INT NOT NULL,
11     category_id INT NOT NULL,
12     PRIMARY KEY (space_id, category_id),
13     FOREIGN KEY (space_id) REFERENCES spaces(id) ON DELETE CASCADE,
14     FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE CASCADE
15 );
16
17 CREATE TABLE IF NOT EXISTS resource_categories (
18     resource_id INT NOT NULL,
19     category_id INT NOT NULL,
20     PRIMARY KEY (resource_id, category_id),
21     FOREIGN KEY (resource_id) REFERENCES resources(id) ON DELETE CASCADE,
22     FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE CASCADE
23 );
24
25 -- Can bang posts (nguoi.sql) da ton tai
26 CREATE TABLE IF NOT EXISTS post_categories (
27     post_id INT NOT NULL,
28     category_id INT NOT NULL,
29     PRIMARY KEY (post_id, category_id),
30     FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,
31     FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE CASCADE
32 );

```

Ràng buộc UNIQUE trên cặp (name, applies_to) cho phép tồn tại hai danh mục cùng tên nhưng khác nhóm đối tượng — ví dụ danh mục “Analog” cho không gian và “Analog” cho bài viết — trong khi vẫn chặn được việc tạo trùng danh mục trong cùng một nhóm.

5.2.3 Phần C — Khuyến mãi

```

1 CREATE TABLE IF NOT EXISTS promotions (
2     id INT AUTO_INCREMENT PRIMARY KEY,
3     provider_id INT NOT NULL,
4     code VARCHAR(50) NOT NULL UNIQUE,
5     description VARCHAR(255),
6     discount_percent TINYINT NOT NULL,
7     start_at DATETIME NOT NULL,
8     end_at DATETIME NOT NULL,
9     max_uses INT DEFAULT NULL,

```

```

10     used_count INT DEFAULT 0,
11     active BOOLEAN DEFAULT TRUE,
12     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
13     CONSTRAINT chk_promo_percent CHECK (discount_percent BETWEEN 1 AND 100),
14     CONSTRAINT chk_promo_time CHECK (end_at > start_at),
15     CONSTRAINT chk_promo_used CHECK (used_count >= 0),
16     FOREIGN KEY (provider_id) REFERENCES users(id) ON DELETE CASCADE
17 );
18
19 -- Gan khuyến mãi vào bookings
20 ALTER TABLE bookings ADD COLUMN promotion_id INT NULL;
21 ALTER TABLE bookings ADD COLUMN discount_amount DECIMAL(10,2) DEFAULT 0;
22 ALTER TABLE bookings ADD CONSTRAINT fk_bookings_promotion
23     FOREIGN KEY (promotion_id) REFERENCES promotions(id) ON DELETE SET NULL;
24 ALTER TABLE bookings ADD CONSTRAINT chk_bookings_discount CHECK (discount_amount >= 0)
    ;

```

Khác với các quan hệ khác trong nhóm, khóa ngoại `bookings.promotion_id` dùng ON DELETE SET NULL thay vì CASCADE. Lý do là dữ liệu đặt chỗ mang giá trị lịch sử và tài chính: khi một chương trình khuyến mãi bị xóa, các đơn đã đặt phải được giữ nguyên, chỉ mất liên kết tới chương trình. Số tiền đã giảm vẫn nằm ở cột `discount_amount` nên báo cáo doanh thu không bị sai lệch.

5.2.4 Phần D — Giao dịch thanh toán

Cột `booking_id` được phép NULL vì không phải giao dịch nào cũng gắn với một lần đặt chỗ: phí đăng ký workshop là một ví dụ. Cột `updated_at` dùng cơ chế ON UPDATE CURRENT_TIMESTAMP để tự động ghi nhận thời điểm cổng thanh toán trả kết quả, không cần tăng ứng dụng can thiệp.

5.2.5 Phần E — Khiếu nại

```

1 CREATE TABLE IF NOT EXISTS complaints (
2     id INT AUTO_INCREMENT PRIMARY KEY,
3     booking_id INT NOT NULL,
4     user_id INT NOT NULL,
5     content TEXT NOT NULL,
6     status ENUM('OPEN', 'IN_REVIEW', 'RESOLVED', 'REJECTED') DEFAULT 'OPEN',
7     resolution TEXT NULL,
8     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
9     resolved_at DATETIME NULL,
10    FOREIGN KEY (booking_id) REFERENCES bookings(id) ON DELETE CASCADE,
11    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
12 );

```

5.2.6 Phần F — Nhật ký trợ lý AI

```
1 CREATE TABLE IF NOT EXISTS chat_logs (  
2     id INT AUTO_INCREMENT PRIMARY KEY,  
3     user_id INT NOT NULL,  
4     session_id VARCHAR(100) NOT NULL,  
5     role ENUM('user', 'assistant', 'system') NOT NULL,  
6     message TEXT NOT NULL,  
7     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
8     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
9 );
```

Việc lưu từng lượt hội thoại thành một dòng riêng, phân biệt bằng cột role, giúp tái dựng chính xác thứ tự đối thoại và thuận tiện cho việc trích xuất dữ liệu huấn luyện hoặc thống kê câu hỏi thường gặp về sau.

5.3 Script ràng buộc — Constraint

5.3.1 Ràng buộc khóa ngoại (FOREIGN KEY)

Bảng	Cột	Tham chiếu	Hành vi khi xóa
service_packages	provider_id	users(id)	CASCADE
package_items	package_id	service_packages(id)	CASCADE
space_categories	space_id / category_id	spaces(id) / categories(id)	CASCADE
resource_categories	resource_id / category_id	resources(id) / categories(id)	CASCADE
post_categories	post_id / category_id	posts(id) / categories(id)	CASCADE
promotions	provider_id	users(id)	CASCADE
bookings	promotion_id	promotions(id)	SET NULL
transactions	booking_id	bookings(id)	SET NULL
transactions	user_id	users(id)	CASCADE
complaints	booking_id / user_id	bookings(id) / users(id)	CASCADE
chat_logs	user_id	users(id)	CASCADE
workshop_registrations	payment_id	transactions(id)	SET NULL

5.3.2 Ràng buộc duy nhất (UNIQUE)

Bảng	Cột / Tổ hợp cột	Mục đích nghiệp vụ
categories	(name, applies_to)	Không tạo trùng danh mục trong cùng nhóm đối tượng
promotions	code	Mã khuyến mãi là duy nhất toàn hệ thống
transactions	(method, external_ref)	Chống ghi trùng callback từ cùng một cổng thanh toán

5.3.3 Ràng buộc miền giá trị (CHECK)

Tên ràng buộc	Điều kiện	Quy tắc nghiệp vụ tương ứng
chk_pkg_price	price >= 0	Giá gói dịch vụ không được âm
chk_pkg_items_qty	quantity > 0	Mỗi thành phần trong gói phải có số lượng dương
chk_promo_percent	discount_percent BETWEEN 1 AND 100	Tỷ lệ giảm giá nằm trong khoảng hợp lệ
chk_promo_time	end_at > start_at	Thời gian kết thúc phải sau thời gian bắt đầu
chk_promo_used	used_count >= 0	Số lượt đã dùng không thể âm
chk_tx_amount	amount > 0	Giao dịch phải có số tiền lớn hơn không
chk_bookings_discount	discount_amount >= 0	Số tiền giảm không được âm

5.3.4 Ràng buộc mặc định (DEFAULT)

Bảng	Cột	Giá trị mặc định và ý nghĩa
promotions	used_count	0 — chương trình mới chưa có lượt dùng
promotions	active	TRUE — mặc định bật ngay khi tạo
transactions	status	'PENDING' — chờ kết quả từ cổng thanh toán
complaints	status	'OPEN' — khiếu nại mới luôn ở trạng thái chờ xử lý
package_items	quantity	1 — mỗi thành phần mặc định một đơn vị
bookings	discount_amount	0 — chưa áp dụng khuyến mãi
Tất cả các bảng	created_at	CURRENT_TIMESTAMP — tự ghi thời điểm khởi tạo

5.4 Trigger nghiệp vụ

5.4.1 Trigger đồng bộ số lượng đăng ký workshop khi đổi trạng thái

Trigger này xử lý tình huống người dùng hủy đăng ký hoặc đăng ký lại. Khi trạng thái chuyển sang CANCELLED, số lượng đăng ký của workshop giảm đi một; khi trạng thái được khôi phục từ CANCELLED sang trạng thái khác, số lượng tăng trở lại.

```
1 DROP TRIGGER IF EXISTS trg_workshop_reg_au $$
2 CREATE TRIGGER trg_workshop_reg_au
3 AFTER UPDATE ON workshop_registrations
4 FOR EACH ROW
5 BEGIN
6     IF OLD.status <> 'CANCELLED' AND NEW.status = 'CANCELLED' THEN
7         UPDATE workshops
8         SET registered_count = GREATEST(registered_count - 1, 0)
9         WHERE id = NEW.workshop_id;
10    ELSEIF OLD.status = 'CANCELLED' AND NEW.status <> 'CANCELLED' THEN
11        UPDATE workshops
12        SET registered_count = registered_count + 1
13        WHERE id = NEW.workshop_id;
14    END IF;
15 END $$
```

Hàm GREATEST(registered_count - 1, 0) đóng vai trò lớp bảo vệ cuối cùng: kể cả khi dữ liệu bị lệch do thao tác bất thường, bộ đếm cũng không bao giờ nhận giá trị âm — điều sẽ vi phạm ràng buộc chk_workshops_registered và làm hỏng toàn bộ giao dịch.

5.4.2 Trigger tăng số lượt dùng khuyến mãi khi tạo đơn đặt chỗ

Khi một đơn đặt chỗ mới được tạo có kèm mã khuyến mãi, trigger tự động tăng bộ đếm used_count của chương trình tương ứng.

```
1 DROP TRIGGER IF EXISTS trg_bookings_promo_ai $$
2 CREATE TRIGGER trg_bookings_promo_ai
3 AFTER INSERT ON bookings
4 FOR EACH ROW
5 BEGIN
6     IF NEW.promotion_id IS NOT NULL THEN
7         UPDATE promotions
8         SET used_count = used_count + 1
9         WHERE id = NEW.promotion_id;
10    END IF;
11 END $$
```

5.4.3 Trigger đồng bộ số lượt dùng khuyến mãi khi sửa đơn đặt chỗ

Đây là trigger phức tạp nhất của nhóm, phải xử lý đủ ba tình huống thay đổi mã khuyến mãi trên một đơn đã tồn tại: thêm mã vào đơn chưa có, gỡ mã khỏi đơn đã có, và đổi từ mã này sang mã khác.

```
1 DROP TRIGGER IF EXISTS trg_bookings_promo_au $$
2 CREATE TRIGGER trg_bookings_promo_au
3 AFTER UPDATE ON bookings
4 FOR EACH ROW
5 BEGIN
6     IF OLD.promotion_id IS NULL AND NEW.promotion_id IS NOT NULL THEN
7         UPDATE promotions SET used_count = used_count + 1 WHERE id = NEW.promotion_id;
8     ELSEIF OLD.promotion_id IS NOT NULL AND NEW.promotion_id IS NULL THEN
9         UPDATE promotions SET used_count = GREATEST(used_count - 1, 0) WHERE id = OLD.
10        promotion_id;
11    ELSEIF OLD.promotion_id IS NOT NULL AND NEW.promotion_id IS NOT NULL
12        AND OLD.promotion_id <> NEW.promotion_id THEN
13        UPDATE promotions SET used_count = GREATEST(used_count - 1, 0) WHERE id = OLD.
14        promotion_id;
15        UPDATE promotions SET used_count = used_count + 1 WHERE id = NEW.
16        promotion_id;
17    END IF;
18 END $$
```

Ba nhánh điều kiện được viết theo thứ tự loại trừ lẫn nhau, bảo đảm mỗi lần cập nhật chỉ kích hoạt đúng một nhánh. Trường hợp mã khuyến mãi không thay đổi, không nhánh nào được thực thi nên bộ đếm giữ nguyên — đây là điểm dễ sai nếu viết bằng hai câu lệnh cộng trừ độc lập.

Tình huống	OLD.promotion_id	NEW.promotion_id	Tác động
Thêm mã vào đơn	NULL	có giá trị	used_count + 1
Gỡ mã khỏi đơn	có giá trị	NULL	used_count - 1
Đổi sang mã khác	mã A	mã B	A giảm 1, B tăng 1
Không đổi mã	mã A	mã A	Không thay đổi

Hạn chế cần ghi nhận: cặp trigger khuyến mãi hiện đếm số đơn đã áp mã chứ chưa tự động chặn khi used_count vượt quá max_uses. Việc kiểm tra hạn mức và thời gian hiệu lực vẫn do tầng ứng dụng thực hiện trước khi ghi đơn. Hướng hoàn thiện được trình bày ở phần kết luận.

5.4.4 Trigger chặn trùng lịch không gian khi tạo đơn đặt chỗ

```

1 CREATE TRIGGER trg_bookings_overlap_bi
2 BEFORE INSERT ON bookings
3 FOR EACH ROW
4 BEGIN
5     DECLARE v_count INT DEFAULT 0;
6     IF NEW.status IN ('PENDING', 'CONFIRMED', 'CHECKED_IN') THEN
7         -- 1a. Check trung booking khac
8         SELECT COUNT(*) INTO v_count
9         FROM bookings
10        WHERE space_id = NEW.space_id
11              AND status IN ('PENDING', 'CONFIRMED', 'CHECKED_IN')
12              AND NEW.start_time < end_time
13              AND NEW.end_time > start_time;
14        IF v_count > 0 THEN
15            SIGNAL SQLSTATE '45000'
16            SET MESSAGE_TEXT = 'Space da co booking trung khung gio';
17        END IF;
18        -- 1b. Check maintenance
19        SELECT COUNT(*) INTO v_count
20        FROM maintenance_schedules
21        WHERE target_type = 'space'
22              AND target_id = NEW.space_id
23              AND status IN ('planned', 'in_progress')
24              AND NEW.start_time < scheduled_end
25              AND NEW.end_time > scheduled_start;
26        IF v_count > 0 THEN
27            SIGNAL SQLSTATE '45000'
28            SET MESSAGE_TEXT = 'Space dang bao tri trong khung gio nay';
29        END IF;
30    END IF;
31 END $$

```

Thời điểm kích hoạt: BEFORE INSERT ON bookings

Mục đích: Ngăn không cho tạo một booking mới bị trùng khung giờ với booking khác trên cùng một không gian, hoặc rơi vào thời gian không gian đang được bảo trì.

Logic xử lý:

- Chỉ kiểm tra khi booking mới có status thuộc {PENDING, CONFIRMED, CHECKED_IN} (booking đã CANCELLED thì không cần chặn).
- Đếm số booking khác cùng space_id, cùng thuộc nhóm status đang hoạt động, có khung giờ giao nhau với booking mới theo điều kiện `NEW.start_time < end_time AND NEW.end_time > start_time` — nếu > 0 thì SIGNAL lỗi 45000 “Space đã có booking trùng khung giờ”.
- Đếm số lịch bảo trì (maintenance_schedules) của cùng space_id đang planned/in_progress có thời gian giao với khung giờ mới — nếu > 0 thì SIGNAL lỗi 45000 “Space đang bảo

trì trong khung giờ này”.

5.4.5 Trigger chặn trùng lịch không gian khi cập nhật đơn đặt chỗ

```
1 CREATE TRIGGER trg_bookings_overlap_bu
2 BEFORE UPDATE ON bookings
3 FOR EACH ROW
4 BEGIN
5     DECLARE v_count INT DEFAULT 0;
6     IF NEW.status IN ('PENDING', 'CONFIRMED', 'CHECKED_IN')
7       AND (OLD.status <=> NEW.status = 0
8           OR OLD.space_id <=> NEW.space_id = 0
9           OR OLD.start_time <=> NEW.start_time = 0
10          OR OLD.end_time <=> NEW.end_time = 0) THEN
11         SELECT COUNT(*) INTO v_count
12         FROM bookings
13         WHERE space_id = NEW.space_id
14            AND id <> NEW.id
15            AND status IN ('PENDING', 'CONFIRMED', 'CHECKED_IN')
16            AND NEW.start_time < end_time
17            AND NEW.end_time > start_time;
18         IF v_count > 0 THEN
19             SIGNAL SQLSTATE '45000'
20             SET MESSAGE_TEXT = 'Space đã có booking trùng khung giờ';
21         END IF;
22         SELECT COUNT(*) INTO v_count
23         FROM maintenance_schedules
24         WHERE target_type = 'space'
25            AND target_id = NEW.space_id
26            AND status IN ('planned', 'in_progress')
27            AND NEW.start_time < scheduled_end
28            AND NEW.end_time > scheduled_start;
29         IF v_count > 0 THEN
30             SIGNAL SQLSTATE '45000'
31             SET MESSAGE_TEXT = 'Space đang bảo trì trong khung giờ này';
32         END IF;
33     END IF;
34 END $$
```

Thời điểm kích hoạt: BEFORE UPDATE ON bookings

Mục đích: Áp dụng lại đúng hai kiểm tra của trigger (1) khi một booking đã tồn tại được cập nhật, đảm bảo việc sửa booking (đổi giờ, đổi không gian, đổi trạng thái) không tạo ra xung đột lịch mới.

Logic xử lý:

- Chỉ chạy kiểm tra khi status mới thuộc nhóm đang hoạt động VÀ có ít nhất một trong

các giá trị status, space_id, start_time, end_time thay đổi so với bản ghi cũ (OLD) — tránh kiểm tra thừa khi cập nhật các cột không liên quan.

- Khi kiểm tra trùng booking khác, loại trừ chính bản ghi đang cập nhật bằng điều kiện `id <> NEW.id`.
- Kiểm tra trùng lịch bảo trì tương tự trigger (1); cùng thông báo lỗi tương ứng.

5.4.6 Trigger chặn trùng lịch thiết bị khi thêm thiết bị vào đơn đặt chỗ

```
1 CREATE TRIGGER trg_booking_resources_overlap_bi
2 BEFORE INSERT ON booking_resources
3 FOR EACH ROW
4 BEGIN
5     DECLARE v_start DATETIME;
6     DECLARE v_end   DATETIME;
7     DECLARE v_count INT DEFAULT 0;
8
9     SELECT start_time, end_time INTO v_start, v_end
10    FROM bookings WHERE id = NEW.booking_id;
11
12    IF v_start IS NULL THEN
13        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Booking không tồn tại';
14    END IF;
15
16    SELECT COUNT(*) INTO v_count
17    FROM booking_resources br
18    JOIN bookings b ON b.id = br.booking_id
19    WHERE br.resource_id = NEW.resource_id
20          AND br.booking_id <> NEW.booking_id
21          AND b.status IN ('PENDING', 'CONFIRMED', 'CHECKED_IN')
22          AND v_start < b.end_time
23          AND v_end   > b.start_time;
24
25    IF v_count > 0 THEN
26        SIGNAL SQLSTATE '45000'
27        SET MESSAGE_TEXT = 'Resource đã được đặt trùng khung giờ';
28    END IF;
29
30    -- Check resource đang bảo trì
31    SELECT COUNT(*) INTO v_count
32    FROM maintenance_schedules
33    WHERE target_type = 'resource'
34          AND target_id = NEW.resource_id
35          AND status IN ('planned', 'in_progress')
36          AND v_start < scheduled_end
```

```

37     AND v_end    > scheduled_start;
38
39     IF v_count > 0 THEN
40         SIGNAL SQLSTATE '45000'
41         SET MESSAGE_TEXT = 'Resource đang bảo trì';
42     END IF;
43 END $$

```

Thời điểm kích hoạt: BEFORE INSERT ON booking_resources

Mục đích: Ngăn không cho một thiết bị (resource) bị gán vào hai booking có khung giờ giao nhau, và chặn khi thiết bị đang trong lịch bảo trì.

Logic xử lý:

- Lấy start_time/end_time của booking tương ứng (NEW.booking_id) từ bảng bookings; nếu không tìm thấy booking → SIGNAL lỗi 45000 “Booking không tồn tại”.
- Đếm các dòng booking_resources khác cùng resource_id (loại trừ chính booking_id hiện tại), có booking liên kết đang ở status {PENDING, CONFIRMED, CHECKED_IN} và khung giờ giao với khung giờ vừa lấy được — nếu > 0 → SIGNAL lỗi 45000 “Resource đã được đặt trùng khung giờ”.
- Đếm lịch bảo trì của cùng resource_id đang planned/in_progress giao với khung giờ đó — nếu > 0 → SIGNAL lỗi 45000 “Resource đang bảo trì”.

5.4.7 Trigger tăng số lượng đăng ký workshop khi thêm lượt đăng ký

```

1 DROP TRIGGER IF EXISTS trg_workshop_reg_ai $$
2 CREATE TRIGGER trg_workshop_reg_ai
3 AFTER INSERT ON workshop_registrations
4 FOR EACH ROW
5 BEGIN
6     UPDATE workshops
7     SET registered_count = registered_count + 1
8     WHERE id = NEW.workshop_id;
9 END $$

```

Mục đích: tăng registered_count lên 1 mỗi khi có một lượt đăng ký mới được ghi nhận thành công.

Logic: UPDATE workshops SET registered_count = registered_count + 1 WHERE id = NEW.workshop_id.

5.4.8 Trigger giảm số lượng đăng ký workshop khi xóa lượt đăng ký

```

1 DROP TRIGGER IF EXISTS trg_workshop_reg_ad $$
2 CREATE TRIGGER trg_workshop_reg_ad
3 AFTER DELETE ON workshop_registrations
4 FOR EACH ROW
5 BEGIN
6     UPDATE workshops
7     SET registered_count = GREATEST(registered_count - 1, 0)
8     WHERE id = OLD.workshop_id;
9 END $$

```

Mục đích: giảm registered_count khi một lượt đăng ký bị xóa cứng khỏi hệ thống.

Logic: UPDATE workshops SET registered_count = GREATEST(registered_count - 1, 0) WHERE id = OLD.workshop_id. Dùng GREATEST(...,0) để tránh số âm nếu có sai lệch dữ liệu.

5.4.9 Trigger chặn đăng ký workshop khi đã đủ chỗ hoặc đã diễn ra

```

1 CREATE TRIGGER trg_workshop_reg_capacity
2 BEFORE INSERT ON workshop_registrations
3 FOR EACH ROW
4 BEGIN
5     DECLARE v_count INT;
6     DECLARE v_cap INT;
7     DECLARE v_ws_date DATETIME;
8
9     SELECT registered_count, capacity, date
10    INTO v_count, v_cap, v_ws_date
11    FROM workshops WHERE id = NEW.workshop_id;
12
13    IF v_count >= v_cap THEN
14        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Workshop da du cho';
15    END IF;
16
17    IF v_ws_date < NOW() THEN
18        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Workshop da dien ra';
19    END IF;
20 END $$

```

Mục đích: chặn đăng ký khi workshop đã đầy hoặc đã diễn ra.

5.5 Chiến lược nạp dữ liệu mẫu

Dữ liệu mẫu được xây dựng trong tệp `sample_data_test.sql`, tuân thủ đúng thứ tự phụ thuộc khóa ngoại: `users` → `spaces` → `resources` → `service_packages` → `package_items` → `promotions` → `bookings` → `booking_resources` và các bảng còn lại. Mỗi lệnh `INSERT` đều kèm mệnh đề `ON DUPLICATE KEY UPDATE` để có thể chạy lại nhiều lần mà không gây lỗi trùng khóa chính, phục vụ việc kiểm thử lặp.

5.6 Index tối ưu truy vấn

Chỉ mục được xây dựng dựa trên các truy vấn nghiệp vụ thực tế đã xác định ở Chương 6, tập trung vào các cột thường xuất hiện trong mệnh đề `WHERE`, `JOIN` và `ORDER BY`. Nguyên tắc chung: ưu tiên chỉ mục ghép theo đúng thứ tự cột mà truy vấn lọc, tránh tạo chỉ mục thừa làm chậm thao tác ghi.

```
1 CREATE INDEX idx_pkg_provider      ON service_packages (provider_id);
2 CREATE INDEX idx_pkg_items_lookup  ON package_items (item_type, item_id);
3 CREATE INDEX idx_promotions_provider_active
4     ON promotions (provider_id, active, start_at, end_at);
5 CREATE INDEX idx_tx_booking        ON transactions (booking_id, status);
6 CREATE INDEX idx_tx_user_time      ON transactions (user_id, created_at);
7 CREATE UNIQUE INDEX uq_tx_external_ref ON transactions (method, external_ref);
8 CREATE INDEX idx_complaints_status ON complaints (status, created_at);
9 CREATE INDEX idx_complaints_user   ON complaints (user_id);
10 CREATE INDEX idx_chat_session      ON chat_logs (session_id, created_at);
11 CREATE INDEX idx_chat_user_time    ON chat_logs (user_id, created_at);
```

Chương 6

KIỂM THỬ VÀ MINH HỌA TRUY VẤN

6.1 Nguyên tắc nạp dữ liệu

Do lược đồ có trigger kiểm tra chéo giữa các bảng, thứ tự nạp không thể tùy ý. Quy tắc áp dụng:

- Nạp theo đúng thứ tự phụ thuộc khóa ngoại: `nguoi1` → `Database_Schema` → `nguoi5` → `nguoi3` → `nguoi2` → `nguoi4` → `triggers`.
- Chỉ tạo trigger SAU khi toàn bộ bảng đã tồn tại, nhưng TRƯỚC khi nạp dữ liệu mẫu — nhờ vậy chính bộ dữ liệu mẫu cũng trở thành một phép kiểm thử: nếu một dòng dữ liệu vi phạm quy tắc nghiệp vụ, nó sẽ bị chặn ngay khi nạp.
- Trong nội bộ tệp dữ liệu, `bookings` được nạp TRƯỚC `maintenance_schedules`. Nếu đảo lại, các booking hợp lệ của tháng 9 sẽ bị trigger `trg_bookings_overlap` bị chặn vì trùng lịch bảo trì — đây là hành vi đúng của trigger chứ không phải lỗi dữ liệu, và tình huống tồn đọng đó được phát hiện lại bằng truy vấn Q22.
- Mọi lệnh `INSERT` đều kèm `ON DUPLICATE KEY UPDATE` để tệp dữ liệu có tính lũy đẳng (idempotent), chạy lại nhiều lần không sinh lỗi trùng khóa.
- Các cột `created_at` của `transactions` và `complaints` được gán giá trị tường minh thay vì để mặc định `CURRENT_TIMESTAMP`, nếu không toàn bộ dữ liệu sẽ dồn vào đúng thời điểm nạp và các truy vấn thống kê theo tháng (Q13, Q25) sẽ mất ý nghĩa.

6.2 Kiểm thử

TC04 — Ranh giới kiểu dữ liệu chặn trước ràng buộc CHECK

Cột `discount_percent` khai báo kiểu `TINYINT` nên chỉ chứa được giá trị tới 127. Khi thử giá trị 150, CSDL báo lỗi 1264 “Out of range” ở tầng kiểu dữ liệu, ràng buộc `chk_promo_percent` chưa kịp được đánh giá. Phải hạ xuống 120 mới quan sát được đúng ràng buộc CHECK.

Đây là ví dụ cho thấy kiểm thử ràng buộc phải chọn giá trị nằm trong miền của kiểu dữ liệu thì mới đo được lớp ràng buộc mong muốn.

```
1 INSERT INTO promotions (provider_id, code, discount_percent, start_at, end_at)
2 VALUES (1, 'SALE120', 120, '2026-10-01', '2026-10-31');
3 -- Lỗi 4025 - vi phạm CHECK chk_promo_percent trên bảng promotions
```

TC15 — Thứ tự kích hoạt giữa trigger và ràng buộc UNIQUE

Kịch bản ban đầu thử đăng ký trùng vào workshop 1 nhưng nhận được thông báo “Workshop đã đủ chỗ” — trigger BEFORE INSERT chạy trước khi ràng buộc UNIQUE được kiểm tra, nên lỗi trả về không phản ánh ràng buộc cần đo. Kịch bản được sửa lại dùng workshop 3 (còn chỗ) mới quan sát được đúng lỗi 1062 trên unique_registration.

```
1 INSERT INTO workshop_registrations (workshop_id, user_id) VALUES (3, 10);
2 -- Lỗi 1062 - trùng giá trị '3-10' trên khóa unique_registration
```

TC20 — Kiểm chứng ON DELETE CASCADE trên hai bảng phụ trách

```
1 INSERT INTO users (id, name, email, password_hash, role)
2 VALUES (99, 'User Tam', 'tam99@example.com', 'hash', 'photographer');
3 INSERT INTO favorites (user_id, space_id) VALUES (99, 1), (99, 2);
4 INSERT INTO notifications (user_id, title, message)
5 VALUES (99, 'Chào mừng', 'Xin chào');
6 DELETE FROM users WHERE id = 99;
7 SELECT
8   (SELECT COUNT(*) FROM favorites WHERE user_id = 99) AS favorites_con_lai,
9   (SELECT COUNT(*) FROM notifications WHERE user_id = 99) AS notifications_con_lai;
```

favorites_con_lai	notifications_con_lai
0	0

Kết quả xác nhận: xóa một tài khoản kéo theo toàn bộ bản ghi favorites và notifications của tài khoản đó, không để lại dữ liệu mồ côi.

6.3 Bộ truy vấn nghiệp vụ minh họa

6.3.1 Nhóm truy vấn người dùng & phân quyền

Q01. Thống kê số lượng tài khoản theo vai trò

```

1 SELECT role,
2     COUNT(*) AS so_luong,
3     ROUND(COUNT(*) * 100.0 / SUM(COUNT(*) OVER ()), 1) AS ty_le_phan_tram
4 FROM users
5 GROUP BY role
6 ORDER BY so_luong DESC;

```

role	so_luong	ty_le_phan_tram
photographer	4	40.0
provider	3	30.0

Nhận xét: Truy vấn dùng hàm cửa sổ `SUM(COUNT(*)) OVER ()` để tính tỷ trọng ngay trong một lần quét, tránh phải truy vấn con lấy tổng số tài khoản.

Q02. Danh sách Provider kèm quy mô đang vận hành

```

1 SELECT u.id,
2     u.business_name,
3     (SELECT COUNT(*) FROM spaces s WHERE s.provider_id = u.id) AS so_phong,
4     (SELECT COUNT(*) FROM resources r JOIN spaces s ON s.id = r.space_id
5     WHERE s.provider_id = u.id) AS so_thiet_bi,
6     (SELECT COALESCE(ROUND(AVG(s.hourly_rate)), 0) FROM spaces s
7     WHERE s.provider_id = u.id) AS gia_gio_trung_binh
8 FROM users u
9 WHERE u.role = 'provider'
10 ORDER BY so_phong DESC;

```

id	business_name	so_phong	so_thiet_bi	gia_gio_trung_binh
1	Studio A	2	3	600000
6	Pixel House	2	3	375000
9	Darkroom Co	1	3	700000

Nhận xét: Dùng truy vấn con tương quan thay vì nhiều `LEFT JOIN` lồng nhau. Nếu `JOIN` trực tiếp `spaces` với `resources` rồi `COUNT`, số phòng sẽ bị nhân bản theo số thiết bị — đây là lỗi “fan-out” thường gặp khi tổng hợp trên nhiều nhánh quan hệ một-nhiều.

Q03. Tài khoản chưa từng phát sinh booking nào

```

1 SELECT u.id, u.name, u.role, u.email
2 FROM users u
3 WHERE NOT EXISTS (SELECT 1 FROM bookings b WHERE b.user_id = u.id)
4 ORDER BY u.role, u.id;

```

id	name	role	email
1	Nguyen Van A	provider	a@example.com
6	Vu Thi F	provider	f@example.com
9	Ngo Van I	provider	i@example.com
7	Dang Van G	expert	g@example.com
4	Pham Thi D	admin	d@example.com

Nhận xét: NOT EXISTS được ưu tiên hơn NOT IN vì an toàn với giá trị NULL và tận dụng được chỉ mục idx_bookings_user. Kết quả cho thấy chỉ các tài khoản provider, expert và admin chưa từng đặt phòng, phù hợp với vai trò của họ trong hệ thống.

6.3.2 Nhóm truy vấn tìm phòng trống

Q04. Tìm phòng còn trống trong khung giờ 2026-09-21 09:00–12:00

```

1 SELECT s.id, s.name, s.type, s.capacity, s.hourly_rate
2 FROM spaces s
3 WHERE s.status = 'AVAILABLE'
4     AND NOT EXISTS (
5     SELECT 1 FROM bookings b
6     WHERE b.space_id = s.id
7           AND b.status IN ('PENDING', 'CONFIRMED', 'CHECKED_IN')
8           AND '2026-09-21 09:00:00' < b.end_time
9           AND '2026-09-21 12:00:00' > b.start_time)
10    AND NOT EXISTS (
11    SELECT 1 FROM maintenance_schedules m
12    WHERE m.target_type = 'space'
13          AND m.target_id = s.id
14          AND m.status IN ('planned', 'in_progress')
15          AND '2026-09-21 09:00:00' < m.scheduled_end
16          AND '2026-09-21 12:00:00' > m.scheduled_start)
17 ORDER BY s.hourly_rate;

```

id	name	type	capacity	hourly_rate
3	Studio Pixel Large	STUDIO	10	400000.00
2	Darkroom A	DARKROOM2		700000.00
4	Darkroom Analog B	DARKROOM3		700000.00

Nhận xét: Đây là truy vấn lõi của toàn hệ thống. Điều kiện chồng lấn thời gian dùng công thức chuẩn (`start_moi < end_cu AND end_moi > start_cu`) — hai khoảng chạm

mép nhau không bị coi là trùng. Studio Mini 1 bị loại vì đã có booking, Studio Pixel Small bị loại vì đang ở trạng thái MAINTENANCE.

Q05. Lịch bận của từng phòng trong tuần 21–27/09/2026

```

1 SELECT s.name AS phong,
2       DATE(b.start_time) AS ngay,
3       TIME(b.start_time) AS gio_bat_dau,
4       TIME(b.end_time) AS gio_ket_thuc,
5       b.status,
6       u.name AS khach
7 FROM bookings b
8 JOIN spaces s ON s.id = b.space_id
9 JOIN users u ON u.id = b.user_id
10 WHERE b.start_time >= '2026-09-21 00:00:00'
11       AND b.start_time < '2026-09-28 00:00:00'
12       AND b.status <> 'CANCELLED'
13 ORDER BY s.name, b.start_time;

```

phong	ngay	gio_bat_dau	gio_ket_thuc	status	khach
Darkroom A	2026-09-22	10:00:00	13:00:00	CONFIRMED	HD Van C
Darkroom A	2026-09-24	15:00:00	18:00:00	PENDING	Bui Thi H
Studio Mini 1	2026-09-21	09:00:00	12:00:00	PENDING	Tran Thi B
Studio Mini 1	2026-09-23	13:00:00	16:00:00	CONFIRMED	HD Dang Van E
Studio Mini 1	2026-09-25	08:00:00	10:00:00	CHECKED IN	MA Thi K

Nhận xét: Truy vấn phục vụ màn hình lịch của provider, kết nối ba bảng và loại các booking đã hủy.

Q06. Tỷ lệ khai thác (giờ đã đặt) của từng phòng trong tháng 09/2026

```

1 SELECT s.id, s.name,
2       COUNT(b.id) AS so_booking,
3       COALESCE(SUM(TIMESTAMPDIFF(HOUR, b.start_time, b.end_time)), 0) AS tong_gio_dat
4       ,
5       ROUND(COALESCE(SUM(TIMESTAMPDIFF(HOUR, b.start_time, b.end_time)), 0)
6             / (30 * 12) * 100, 2) AS ty_le_khai_thac
7 FROM spaces s

```

```

7 LEFT JOIN bookings b
8   ON b.space_id = s.id
9   AND b.status IN ('CONFIRMED', 'CHECKED_IN', 'COMPLETED', 'PENDING')
10  AND b.start_time >= '2026-09-01' AND b.start_time < '2026-10-01'
11 GROUP BY s.id, s.name
12 ORDER BY tong_gio_dat DESC;

```

id	name	so_booking	tong_gio_dat	ty_le_khai_thac
1	Studio Mini 1	3	8	2.22
2	Darkroom A	2	6	1.67
3	Studio Pixel Large	1	3	0.83
4	Darkroom Analog B	1	3	0.83
5	Studio Pixel Small	0	0	0.00

Nhận xét: Mẫu số 30×12 là số giờ vận hành tối đa trong tháng (30 ngày $\times$ 12 giờ/ngày). LEFT JOIN kèm điều kiện lọc đặt trong mệnh đề ON chứ không phải WHERE, nhờ đó phòng chưa có booking nào vẫn xuất hiện với giá trị 0.

6.3.3 Nhóm truy vấn gói dịch vụ

Q07. Chi tiết thành phần của từng gói dịch vụ

```

1 SELECT p.id,
2        p.name AS goi_dich_vu,
3        p.price AS gia_goi,
4        GROUP_CONCAT(CONCAT(pi.item_type, '#', pi.item_id, ' x', pi.quantity)
5                      ORDER BY pi.item_type, pi.item_id SEPARATOR ' | ') AS thanh_phan
6 FROM service_packages p
7 LEFT JOIN package_items pi ON pi.package_id = p.id
8 GROUP BY p.id, p.name, p.price;

```

id	goi_dich_vu	gia_goi	thanh_phan
1	Gói Chụp Studio	1800000.00	space#1 x1 resource#1 x1 resource#2 x1
2	Gói Darkroom	1200000.00	space#2 x1 resource#3 x1

Nhận xét: package_items dùng khóa đa hình (item_type, item_id) nên không thể JOIN trực tiếp. GROUP_CONCAT được dùng để gộp thành phần của gói thành một dòng dễ đọc.

Q08. So sánh giá gói với tổng giá thuê lẻ 3 giờ

```
1 SELECT p.id, p.name AS goi_dich_vu, p.price AS gia_goi,
2       COALESCE(sp.gia_phong, 0) * 3 + COALESCE(rs.gia_thiet_bi, 0) AS gia_le_3_gio,
3       p.price - (COALESCE(sp.gia_phong, 0) * 3 + COALESCE(rs.gia_thiet_bi, 0)) AS
4       chenh_lech
5 FROM service_packages p
6 LEFT JOIN (SELECT pi.package_id, SUM(s.hourly_rate * pi.quantity) AS gia_phong
7            FROM package_items pi JOIN spaces s ON s.id = pi.item_id
8            WHERE pi.item_type = 'space' GROUP BY pi.package_id) sp ON sp.package_id =
9            p.id
10 LEFT JOIN (SELECT pi.package_id, SUM(r.rental_price * pi.quantity) AS gia_thiet_bi
11            FROM package_items pi JOIN resources r ON r.id = pi.item_id
12            WHERE pi.item_type = 'resource' GROUP BY pi.package_id) rs ON rs.package_id =
13            p.id
14 ORDER BY p.id;
```

id	goi_dich_vu	gia_goi	gia_le_3_gio	chenh_lech
1	Gói Chụp Studio	1800000.00	1950000.00	-150000.00
2	Gói Darkroom	1200000.00	2300000.00	-1100000.00

Nhận xét: Hai truy vấn con tách riêng theo item_type là cách xử lý khóa đa hình an toàn: nếu JOIN chung, item_id = 1 sẽ khớp nhầm cả spaces(1) lẫn resources(1). Kết quả cho thấy cả hai gói đều rẻ hơn thuê lẻ, đúng định hướng khuyến khích khách mua gói.

Q09. Gói dịch vụ được đặt nhiều nhất kèm doanh thu

```
1 SELECT p.name AS goi_dich_vu,
2       COUNT(b.id) AS so_lan_dat,
3       COALESCE(SUM(b.total_price - b.discount_amount), 0) AS doanh_thu_thuc
4 FROM service_packages p
5 LEFT JOIN bookings b ON b.package_id = p.id AND b.status <> 'CANCELLED'
6 GROUP BY p.id, p.name
7 ORDER BY so_lan_dat DESC, doanh_thu_thuc DESC;
```

goi_dich_vu	so_lan_dat	doanh_thu_thuc
Gói Darkroom	2	4500000.00
Gói Chụp Studio	2	2850000.00

Nhận xét: Doanh thu được tính trên total_price trừ discount_amount để phản ánh số tiền thực thu.

6.3.4 Nhóm truy vấn tài nguyên & tồn kho

Q10. Phân bố thiết bị theo danh mục và trạng thái

```
1 SELECT category,
2     COUNT(*) AS tong,
3     SUM(status = 'AVAILABLE') AS san_sang,
4     SUM(status = 'MAINTENANCE') AS bao_tri,
5     SUM(status = 'DAMAGED') AS hu_hong
6 FROM resources
7 GROUP BY category
8 ORDER BY tong DESC;
```

category	tong	san_sang	bao_tri	hu_hong
LENS	2	1	1	0
CAMERA	2	2	0	0
PAPER	1	1	0	0
ENLARGER	1	1	0	0
SCANNER	1	1	0	0
LIGHTING	1	1	0	0
CHEMICAL	1	1	0	0

Nhận xét: Kỹ thuật SUM(điều_kiện) tận dụng việc biểu thức logic trả về 1/0, cho phép đếm nhiều nhóm trạng thái trong một lần quét bảng thay vì dùng nhiều truy vấn con.

Q11. Cảnh báo tồn kho thấp (ngưỡng khác nhau cho vật tư và thiết bị)

```
1 SELECT r.id, r.name, r.category, r.stock_quantity, r.unit,
2     s.name AS thuoc_phong,
3     CASE WHEN r.category IN ('CHEMICAL', 'PAPER') THEN 'Vat tu tieu hao'
4     ELSE 'Thiet bi' END AS nhom
5 FROM resources r
6 JOIN spaces s ON s.id = r.space_id
7 WHERE r.stock_quantity IS NOT NULL
8     AND ( (r.category IN ('CHEMICAL', 'PAPER') AND r.stock_quantity < 10)
9     OR (r.category NOT IN ('CHEMICAL', 'PAPER') AND r.stock_quantity < 2) )
10 ORDER BY r.stock_quantity;
```

id	name	category	stock_quantity	unit	thuoc_phong	nhom
6	Epson V600 Scanner	SCANNER	1	unit	Darkroom Analog B	Thiet bi
4	Nikon FM2	CAMERA	1	unit	Studio Pixel Large	Thiet bi

3	Enlarger Durst	ENLARGER	1	unit	Darkroom A	Thiết bị
7	Kodak D-76 Developer	CHEMICAL	8	lit	Darkroom Analog B	Vat tu tiêu hao

Nhận xét: Ngưỡng cảnh báo khác nhau theo bản chất tài nguyên: vật tư tiêu hao (hóa chất, giấy ảnh) cảnh báo khi dưới 10 đơn vị, còn thiết bị cảnh báo khi chỉ còn dưới 2 chiếc.

Q12. Thiết bị được thuê nhiều nhất

```

1 SELECT r.id, r.name, r.category,
2        COUNT(b.id) AS so_luot_thue,
3        COALESCE(SUM(br.quantity), 0) AS tong_so_luong
4 FROM resources r
5 LEFT JOIN booking_resources br ON br.resource_id = r.id
6 LEFT JOIN bookings b ON b.id = br.booking_id AND b.status <> 'CANCELLED'
7 GROUP BY r.id, r.name, r.category
8 HAVING so_luot_thue > 0
9 ORDER BY so_luot_thue DESC, tong_so_luong DESC;
```

id	name	category	so_luot_thue	tong_so_luong
2	50mm Lens	LENS	2	2
3	Enlarger Durst	ENLARGER	2	2
4	Nikon FM2	CAMERA	2	2
6	Epson V600 Scanner	SCANNER	2	2
1	Canon 5D Mark IV	CAMERA	2	2
5	Godox Lighting Kit	LIGHTING	1	2
7	Kodak D-76 Developer	CHEMICAL	1	2

Nhận xét: Đếm b.id thay vì br.booking_id để các booking đã hủy không bị tính vào lượt thuê.

6.3.5 Nhóm truy vấn giao dịch và doanh thu

Q13. Doanh thu theo tháng (chỉ tính giao dịch SUCCESS)

```

1 SELECT DATE_FORMAT(t.created_at, '%Y-%m') AS thang,
```

```

2      COUNT(*) AS so_giao_dich,
3      SUM(t.amount) AS doanh_thu
4 FROM transactions t
5 WHERE t.status = 'SUCCESS'
6 GROUP BY DATE_FORMAT(t.created_at, '%Y-%m')
7 ORDER BY thang;

```

thang	so_giao_dich	doanh_thu
2026-08	4	5200000.00
2026-09	5	5450000.00

Nhận xét: Chỉ giao dịch SUCCESS được tính vào doanh thu; các giao dịch PENDING, FAILED, REFUNDED bị loại.

Q14. Doanh thu theo Provider

```

1 SELECT u.id, u.business_name,
2      COUNT(DISTINCT b.id) AS so_booking,
3      COALESCE(SUM(t.amount), 0) AS doanh_thu
4 FROM users u
5 JOIN spaces s ON s.provider_id = u.id
6 JOIN bookings b ON b.space_id = s.id
7 LEFT JOIN transactions t ON t.booking_id = b.id AND t.status = 'SUCCESS'
8 WHERE u.role = 'provider'
9 GROUP BY u.id, u.business_name
10 ORDER BY doanh_thu DESC;

```

id	business_name	so_booking	doanh_thu
1	Studio A	7	5500000.00
6	Pixel House	3	2800000.00
9	Darkroom Co	2	1400000.00

Nhận xét: Chuỗi JOIN users → spaces → bookings → transactions cho phép quy doanh thu về đúng chủ sở hữu phòng.

Q15. Tỷ lệ thành công theo phương thức thanh toán

```

1 SELECT method,
2      COUNT(*) AS tong_giao_dich,
3      SUM(status = 'SUCCESS') AS thanh_cong,
4      SUM(status = 'FAILED') AS that_bai,
5      ROUND(SUM(status = 'SUCCESS') * 100.0 / COUNT(*), 1) AS ty_le_thanh_cong
6 FROM transactions

```

```

7 GROUP BY method
8 ORDER BY ty_le_thanh_cong DESC;

```

method	tong_giao_dich	hanh_cong	that_bai	ty_le_thanh_cong
MOMO	3	3	0	100.0
CASH	1	1	0	100.0
VNPAY	6	4	2	66.7
STRIPE	2	1	1	50.0

Nhận xét: Chỉ số vận hành quan trọng: STRIPE có tỷ lệ thành công thấp nhất (50%), cần theo dõi thêm khi khối lượng giao dịch tăng.

Q16. Booking đã xác nhận nhưng chưa có giao dịch thành công (cảnh báo công nợ)

```

1 SELECT b.id, u.name AS khách, s.name AS phong, b.start_time,
2        b.total_price - b.discount_amount AS so_tien_phai_thu,
3        COALESCE(MAX(t.status), 'CHUA_CO_GIAO_DICH') AS trang_thai_gd
4 FROM bookings b
5 JOIN users u ON u.id = b.user_id
6 JOIN spaces s ON s.id = b.space_id
7 LEFT JOIN transactions t ON t.booking_id = b.id
8 WHERE b.status IN ('PENDING', 'CONFIRMED', 'CHECKED_IN')
9 GROUP BY b.id, u.name, s.name, b.start_time, b.total_price, b.discount_amount
10 HAVING SUM(t.status = 'SUCCESS') IS NULL OR SUM(t.status = 'SUCCESS') = 0
11 ORDER BY b.start_time;

```

id	khách	phong	start_time	so_tien_phai_thu	trang_thai_gd
1	Tran Thi B	Studio Mini 1	2026-09-21 09:00:00	1350000.00	PENDING
4	Bui Thi H	Darkroom A	2026-09-24 15:00:00	2400000.00	FAILED
10	Tran Thi B	Darkroom Analog B	2026-09-28 09:00:00	2100000.00	CHUA_CO_GIAO_DICH
11	Bui Thi H	Studio Pixel Large	2026-09-29 10:00:00	1200000.00	CHUA_CO_GIAO_DICH

Nhận xét: Truy vấn cảnh báo công nợ, phân biệt rõ hai tình huống: đã có giao dịch nhưng chưa thành công (booking 1, 4) và hoàn toàn chưa phát sinh giao dịch (booking 10, 11).

6.3.6 Nhóm truy vấn cộng đồng

Q17. Xếp hạng phòng theo điểm đánh giá trung bình

```
1 SELECT s.id, s.name, s.type,  
2       COUNT(rv.id) AS so_danh_gia,  
3       ROUND(AVG(rv.rating), 2) AS diem_trung_binh  
4 FROM spaces s  
5 JOIN reviews rv ON rv.target_type = 'studio' AND rv.target_id = s.id  
6 GROUP BY s.id, s.name, s.type  
7 ORDER BY diem_trung_binh DESC, so_danh_gia DESC;
```

id	name	type	so_danh_gia	diem_trung_binh
2	Darkroom A	DARKROOM1		5.00
1	Studio Mini 1	STUDIO	2	4.50
3	Studio Pixel Large	STUDIO	1	3.00

Nhận xét: reviews dùng quan hệ đa hình qua (target_type, target_id) nên điều kiện JOIN phải kèm ràng buộc target_type = 'studio' để không lấy nhầm đánh giá của workshop hay gói dịch vụ.

Q18. Tình hình workshop: tỷ lệ lấp đầy và số người đã thanh toán

```
1 SELECT w.id, w.title, w.date, w.capacity, w.registered_count,  
2       ROUND(w.registered_count * 100.0 / w.capacity, 1) AS ty_le_lap_day,  
3       SUM(wr.status = 'PAID') AS da_thanh_toan  
4 FROM workshops w  
5 LEFT JOIN workshop_registrations wr ON wr.workshop_id = w.id  
6 GROUP BY w.id, w.title, w.date, w.capacity, w.registered_count  
7 ORDER BY ty_le_lap_day DESC;
```

id	title	date	capacity	registered_count	ty_le_lap_day	da_thanh_toan
1	Workshop Chụp Hình Màu	2026-10-10 09:00:00	3	3	100.0	1
2	Workshop Film Analog	2026-10-12 14:00:00	2	1	50.0	1
3	Advanced Retouching	2026-10-15 10:00:00	4	1	25.0	0

Nhận xét: Đối chiếu registered_count (được trigger đồng bộ tự động) với số đăng ký thực tế — một phép kiểm tra chéo tính nhất quán của dữ liệu.

Q19. Phòng được yêu thích nhất và tỷ lệ chuyển đổi sang booking

```

1 SELECT s.id, s.name,
2       COUNT(DISTINCT f.user_id) AS so_luot_yeu_thich,
3       COUNT(DISTINCT b.user_id) AS so_khach_da_dat,
4       ROUND(COUNT(DISTINCT b.user_id) * 100.0
5             / NULLIF(COUNT(DISTINCT f.user_id), 0), 1) AS ty_le_chuyen_doi
6 FROM spaces s
7 LEFT JOIN favorites f ON f.space_id = s.id
8 LEFT JOIN bookings b ON b.space_id = s.id AND b.user_id = f.user_id
9                      AND b.status <> 'CANCELLED'
10 GROUP BY s.id, s.name
11 ORDER BY so_luot_yeu_thich DESC;

```

id	name	so_luot_yeu_thich	so_khach_da_dat	ty_le_chuyen_doi
1	Studio Mini 1	4	3	75.0
2	Darkroom A	3	2	66.7
3	Studio Pixel Large	3	2	66.7
4	Darkroom Analog B	1	1	100.0
5	Studio Pixel Small	0	0	NULL

Nhận xét: Truy vấn trên bảng favorites thuộc phần phụ trách. Tỷ lệ chuyển đổi từ lượt yêu thích sang lượt đặt thật là chỉ số đo hiệu quả của tính năng. NULLIF được dùng để tránh lỗi chia cho 0 ở phòng chưa có ai yêu thích.

Q20. Thông báo chưa đọc của từng người dùng, phân loại theo nhóm

```

1 SELECT u.id, u.name,
2       COUNT(*) AS tong_thong_bao,
3       SUM(n.is_read = FALSE) AS chua_doc,
4       GROUP_CONCAT(DISTINCT n.type ORDER BY n.type SEPARATOR ', ') AS cac_loai
5 FROM notifications n
6 JOIN users u ON u.id = n.user_id
7 GROUP BY u.id, u.name
8 HAVING chua_doc > 0
9 ORDER BY chua_doc DESC;

```

id	name	tong_thong_bao	chua_doc	cac_loai
----	------	----------------	----------	----------

8	Bui Thi H	3	3	payment, workshop, maintenance
5	Hoang Van E	2	2	payment, review
2	Tran Thi B	3	1	booking, complaint
3	Le Van C	2	1	payment, workshop
10	Mai Thi K	2	1	maintenance, system

Nhận xét: Truy vấn trên bảng notifications thuộc phần phụ trách, khai thác đúng chỉ mục idx_notifications_user_unread. Mệnh đề HAVING lọc bỏ những người đã đọc hết thông báo.

6.3.7 Nhóm truy vấn vận hành & bảo trì

Q21. Lịch bảo trì sắp tới kèm tên đối tượng (polymorphic join)

```

1 SELECT m.id, m.target_type, m.target_id,
2        COALESCE(s.name, r.name) AS ten_doi_tuong,
3        m.scheduled_start, m.scheduled_end, m.status, m.reason
4 FROM maintenance_schedules m
5 LEFT JOIN spaces s ON m.target_type = 'space' AND s.id = m.target_id
6 LEFT JOIN resources r ON m.target_type = 'resource' AND r.id = m.target_id
7 WHERE m.status IN ('planned', 'in_progress')
8 ORDER BY m.scheduled_start;
```

id	target_type	target_id	ten_doi_tuong	scheduled_start	scheduled_end	status	reason
1	space	1	Studio Mini 1	2026-09-21 10:00:00	2026-09-21 11:00:00	planned	Bảo trì hệ thống ánh sáng
2	resource	1	Canon 5D Mark IV	2026-09-22 11:00:00	2026-09-22 12:00:00	in_progress	Bảo trì máy ảnh
3	space	2	Darkroom A	2026-09-24 16:00:00	2026-09-24 17:30:00	planned	Kiểm tra phòng tối

Nhận xét: Kỹ thuật JOIN có điều kiện theo target_type kết hợp COALESCE là cách chuẩn để “giải” một khóa đa hình thành tên đối tượng cụ thể trong một truy vấn duy nhất.

Q22. Phát hiện booking đang xung đột với lịch bảo trì đã lên sau đó

```

1 SELECT b.id AS booking_id, s.name AS phong, b.start_time, b.end_time,
2        m.id AS maintenance_id, m.scheduled_start, m.scheduled_end, m.reason
3 FROM bookings b
```

```

4 JOIN spaces s ON s.id = b.space_id
5 JOIN maintenance_schedules m
6     ON m.target_type = 'space' AND m.target_id = b.space_id
7     AND m.status IN ('planned', 'in_progress')
8     AND b.start_time < m.scheduled_end
9     AND b.end_time > m.scheduled_start
10 WHERE b.status IN ('PENDING', 'CONFIRMED', 'CHECKED_IN')
11 ORDER BY b.start_time;

```

	booking_id	start_time	end_time	maint._id	scheduled_start	scheduled_end	reason
1	Studio Mini 1	2026-09-21 09:00:00	2026-09-21 12:00:00	1	2026-09-21 10:00:00	2026-09-21 11:00:00	Bảo trì hệ thống ánh sáng
4	Darkroom A	2026-09-24 15:00:00	2026-09-24 18:00:00	3	2026-09-24 16:00:00	2026-09-24 17:30:00	Kiểm tra phòng tối

Nhận xét: Truy vấn giám sát quan trọng. Trigger chỉ chặn được booking đặt sau khi lịch bảo trì đã tồn tại; nếu lịch bảo trì được lập sau, xung đột vẫn phát sinh. Truy vấn này phát hiện 2 trường hợp tồn đọng như vậy để bộ phận vận hành chủ động liên hệ khách hàng.

Q23. Khiếu nại đang tồn đọng và thời gian chờ xử lý

```

1 SELECT c.id, u.name AS nguoi_khieu_nai, c.booking_id, c.status, c.created_at,
2     TIMESTAMPDIFF(DAY, c.created_at, NOW()) AS so_ngay_cho,
3     LEFT(c.content, 50) AS trich_noi_dung
4 FROM complaints c
5 JOIN users u ON u.id = c.user_id
6 WHERE c.status IN ('OPEN', 'IN_REVIEW')
7 ORDER BY so_ngay_cho DESC;

```

id	nguoi_khieu_nai	booking_id	status	created_at	so_ngay_cho	trich_noi_dung
3	Mai Thi K	9	OPEN	2026-08-26 09:30:00	22	Phòng không được dọn sạch trước giờ nhận
1	Bui Thi H	4	IN_REVIEW	2026-09-10 08:00:00	7	Thanh toán bị tru tiền nhưng booking vẫn ở trạng thái pending

Nhận xét: Dùng TIMESTAMPDIFF để đo thời gian chờ xử lý, phục vụ cam kết SLA với khách hàng.

6.3.8 Nhóm truy vấn nâng cao

Q24. Xếp hạng phòng theo doanh thu trong từng Provider (RANK OVER PARTITION)

```
1 WITH doanh_thu_phong AS (  
2     SELECT s.provider_id, s.id AS space_id, s.name,  
3           COALESCE(SUM(b.total_price - b.discount_amount), 0) AS doanh_thu  
4     FROM spaces s  
5     LEFT JOIN bookings b ON b.space_id = s.id AND b.status <> 'CANCELLED'  
6     GROUP BY s.provider_id, s.id, s.name  
7 )  
8 SELECT u.business_name, d.name AS phong, d.doanh_thu,  
9       RANK() OVER (PARTITION BY d.provider_id ORDER BY d.doanh_thu DESC) AS  
10      hang_trong_provider  
11 FROM doanh_thu_phong d  
12 JOIN users u ON u.id = d.provider_id  
13 ORDER BY u.business_name, hang_trong_provider;
```

business_namephong		doanh_thu	hang_trong_provider
Darkroom Co	Darkroom	3500000.00	1
	Analog B		
Pixel House	Studio Pixel	4000000.00	1
	Large		
Pixel House	Studio Pixel	0.00	2
	Small		
Studio A	Studio Mini 1	4750000.00	1
Studio A	Darkroom A	4500000.00	2

Nhận xét: Kết hợp CTE và RANK() OVER (PARTITION BY ...) để xếp hạng trong nội bộ từng provider. Nếu dùng GROUP BY thuần, phải viết truy vấn tương quan phức tạp hơn nhiều mới đạt kết quả tương đương.

Q25. Doanh thu lũy kế theo tháng (running total)

```
1 WITH thang AS (  
2     SELECT DATE_FORMAT(created_at, '%Y-%m') AS ky, SUM(amount) AS doanh_thu  
3     FROM transactions WHERE status = 'SUCCESS'  
4     GROUP BY DATE_FORMAT(created_at, '%Y-%m')  
5 )  
6 SELECT ky, doanh_thu,  
7       SUM(doanh_thu) OVER (ORDER BY ky) AS luy_ke,  
8       ROUND(AVG(doanh_thu) OVER (ORDER BY ky ROWS BETWEEN 1 PRECEDING AND CURRENT ROW  
9       ), 0) AS trung_binh_truot_2ky
```

```
9 FROM thang;
```

ky	doanh_thu	luy_ke	trung_binh_truot_2ky
2026-08	5200000.00	5200000.00	5200000
2026-09	5450000.00	10650000.00	5325000

Nhận xét: Running total bằng SUM() OVER (ORDER BY ...) kèm trung bình trượt hai kỳ qua khung cửa sổ ROWS BETWEEN 1 PRECEDING AND CURRENT ROW.

Q26. Phân hạng khách hàng theo tổng chi tiêu (NTILE)

```
1 WITH chi_tieu AS (
2     SELECT u.id, u.name,
3           COUNT(b.id) AS so_booking,
4           COALESCE(SUM(b.total_price - b.discount_amount), 0) AS tong_chi
5     FROM users u
6     JOIN bookings b ON b.user_id = u.id AND b.status <> 'CANCELLED'
7     WHERE u.role = 'photographer'
8     GROUP BY u.id, u.name
9 )
10 SELECT name, so_booking, tong_chi,
11        NTILE(3) OVER (ORDER BY tong_chi DESC) AS hang_khach,
12        ROUND(tong_chi * 100.0 / SUM(tong_chi) OVER (), 1) AS ty_trong_phan_tram
13 FROM chi_tieu
14 ORDER BY tong_chi DESC;
```

name	so_booking	tong_chi	hang_khach	ty_trong_phan_tram
Bui Thi H	3	5000000.00	1	34.1
Tran Thi B	3	4650000.00	1	31.7
Hoang Van E	2	3100000.00	2	21.2
Mai Thi K	2	1900000.00	3	13.0

Nhận xét: NTILE(3) chia khách hàng thành ba nhóm chi tiêu, phục vụ chính sách khuyến mãi phân tầng. Hai khách hàng nhóm 1 chiếm gần 66% tổng chi tiêu.

Q27. Khoảng cách giữa hai lần đặt phòng liên tiếp của mỗi khách (LAG)

```
1 SELECT u.name AS khách, b.id AS booking_id, DATE(b.start_time) AS ngay_dat,
2        DATE(LAG(b.start_time) OVER (PARTITION BY b.user_id ORDER BY b.start_time)) AS
   lan_truoc,
3        DATEDIFF(b.start_time,
4                 LAG(b.start_time) OVER (PARTITION BY b.user_id ORDER BY b.start_time))
   AS cach_bao_nhieu_ngay
```

```

5 FROM bookings b
6 JOIN users u ON u.id = b.user_id
7 WHERE b.status <> 'CANCELLED'
8 ORDER BY u.name, b.start_time;

```

khach	booking_id	ngay_dat	lan_truoc	cach_bao_nhieu_ngay
Bui Thi H	8	2026-08-20	NULL	NULL
Bui Thi H	4	2026-09-24	2026-08-20	35
Bui Thi H	11	2026-09-29	2026-09-24	5
Hoang Van E	7	2026-08-12	NULL	NULL
Hoang Van E	3	2026-09-23	2026-08-12	42
Le Van C	2	2026-09-22	NULL	NULL
Mai Thi K	9	2026-08-25	NULL	NULL
Mai Thi K	5	2026-09-25	2026-08-25	31
Tran Thi B	6	2026-08-05	NULL	NULL
Tran Thi B	1	2026-09-21	2026-08-05	47
Tran Thi B	10	2026-09-28	2026-09-21	7

Nhận xét: LAG() lấy mốc thời gian của lần đặt liền trước trong cùng một khách hàng, từ đó tính chu kỳ quay lại — dữ liệu đầu vào cho bài toán dự báo và nhắc lịch tự động.

6.4 Kiểm thử trigger

Hệ thống có 9 trigger cài đặt các quy tắc nghiệp vụ không thể diễn đạt bằng ràng buộc khai báo. Mục này kiểm thử từng trigger theo hai hướng: hướng phủ định (thao tác sai phải bị chặn) và hướng khẳng định (thao tác hợp lệ phải qua, và tác dụng phụ phải xảy ra đúng).

6.4.1 Phân tích một số trường hợp đáng chú ý

TG02 — Kiểm thử giá trị biên của điều kiện chồng lấn

Booking cũ kết thúc lúc 12:00, booking mới bắt đầu đúng 12:00. Do trigger dùng phép so sánh nghiêm ngặt (`NEW.start_time < end_time AND NEW.end_time > start_time`), hai khoảng chạm mép không bị coi là trùng nên lệnh được chấp nhận. Nếu dùng `<=` và `>=`, hệ thống sẽ từ chối sai các ca đặt liên tiếp — một lỗi nghiệp vụ nghiêm trọng vì nó làm giảm công suất khai thác phòng.

```

1 INSERT INTO bookings (user_id, space_id, start_time, end_time, total_price,
   qr_code_hash)
2 VALUES (8, 1, '2026-09-21 12:00:00', '2026-09-21 14:00:00', 1000000, 'QR-TG02');

```

```

3 SELECT id, space_id, start_time, end_time FROM bookings WHERE qr_code_hash = 'QR-TG02'
;

```

id	space_id	start_time	end_time
20	1	2026-09-21 12:00:00	2026-09-21 14:00:00

TG03 — Thứ tự kiểm tra bên trong trigger ảnh hưởng tới thông báo lỗi

Kịch bản ban đầu chọn một phòng vừa có booking trùng giờ vừa có lịch bảo trì, kết quả trả về thông báo “Space đã có booking trùng khung giờ” vì nhánh kiểm tra booking đứng trước nhánh kiểm tra bảo trì. Để đo đúng nhánh bảo trì, kịch bản được dựng lại trên một phòng trống hoàn toàn rồi mới thêm lịch bảo trì.

```

1 INSERT INTO maintenance_schedules
2     (target_type, target_id, scheduled_start, scheduled_end, reason, status,
3      created_by)
4 VALUES ('space', 3, '2026-10-02 08:00:00', '2026-10-02 12:00:00',
5         'Ve sinh dinh ky', 'planned', 4);
6
7 INSERT INTO bookings (user_id, space_id, start_time, end_time, total_price)
8 VALUES (8, 3, '2026-10-02 09:00:00', '2026-10-02 11:00:00', 800000);
9
10 -- Loi 1644 - Space đang bảo trì trong khung giờ này

```

TG06 và TG08 — Hai nhánh khác nhau của cùng một trigger

Cùng trigger trg_booking_resources_overlap_bi nhưng hai kịch bản chạm vào hai nhánh khác nhau: TG06 chạm nhánh “thiết bị đã được booking khác giữ trong khung giờ giao nhau”, TG08 chạm nhánh “thiết bị nằm trong lịch bảo trì”. Thông báo lỗi khác nhau giúp tăng ứng dụng hiển thị đúng nguyên nhân cho người dùng cuối.

```

1 -- TG06: Loi 1644 - Resource đã được đặt trong khung giờ
2 -- TG08: Loi 1644 - Resource đang bảo trì

```

TG16 — Tác dụng phụ kép trong một lệnh UPDATE

Khi khách đổi từ mã WELCOME10 sang VIP20, trigger AFTER UPDATE phải thực hiện đồng thời hai phép cập nhật ngược chiều trên bảng promotions. Kết quả xác nhận cả hai bộ đếm đều được điều chỉnh đúng trong cùng một giao dịch.

```

1 SELECT
2     (SELECT used_count FROM promotions WHERE id = 1) AS promo1_truoc,
3     (SELECT used_count FROM promotions WHERE id = 2) AS promo2_truoc;
4
5 UPDATE bookings SET promotion_id = 2 WHERE id = 5;

```

```

6
7 SELECT
8   (SELECT used_count FROM promotions WHERE id = 1) AS promo1_sau,
9   (SELECT used_count FROM promotions WHERE id = 2) AS promo2_sau;

```

promo1_truoc	promo2_truoc
2	1
promo1_sau	promo2_sau
1	2

6.4.2 Hạn chế đã phát hiện qua kiểm thử

Quá trình kiểm thử làm lộ ra một số điểm cần lưu ý, được ghi nhận để bổ sung vào mục Hạn chế của Chương 7:

- **Trigger chỉ kiểm tra tại thời điểm ghi booking.** Nếu lịch bảo trì được lập SAU khi booking đã tồn tại, xung đột không bị phát hiện tự động — hiện phải rà soát bằng truy vấn Q22. Hướng khắc phục là bổ sung trigger BEFORE INSERT trên maintenance_schedules để chặn hoặc cảnh báo ngay khi lập lịch.
- **Trigger chống trùng lịch dựa trên phép đếm (SELECT COUNT rồi SIGNAL).** Trong môi trường nhiều giao dịch đồng thời, hai phiên có thể cùng đọc kết quả 0 rồi cùng ghi thành công. Với tải hiện tại của đồ án rủi ro là chấp nhận được; ở môi trường thật cần khóa hàng (SELECT ... FOR UPDATE) hoặc chỉ mục loại trừ khoảng thời gian.
- **Cột bookings.space_id được khai báo NOT NULL,** nên khi khách đặt trọn gói nhiều phòng, ứng dụng phải chọn một phòng làm phòng chính. Đây là đánh đổi có chủ đích để giữ khóa ngoại đơn giản, nhưng khiến truy vấn doanh thu theo phòng (Q14, Q24) quy toàn bộ giá trị gói về phòng chính.
- **registered_count của workshops là dữ liệu dẫn xuất** được trigger đồng bộ. Nếu trigger bị vô hiệu hóa trong một lần bảo trì, giá trị này sẽ lệch so với thực tế; truy vấn Q18 được dùng như một phép đối chiếu định kỳ.